

基于两阶段随机规划与滚动时域优化的微网经济调度模型

摘 要

并网型微网可在电价低谷时向储能充电、在电价高峰时由储能放电，把电量从低价时段转移到高价时段，以更低的购电费用满足小区负载。本文围绕购电与储能充放电策略的制定，从确定性的单日调度出发，逐步引入负荷与光伏的不确定性、日内预报更新与波动电价建模求解，并在全年 334 天、每天 144 个时段上完成回测。

针对问题一，购电问题的本质是利用储能在低价时段充电、高价时段放电，转移电量以降低费用。储能充放电互斥若用 0-1 变量刻画，模型将变为混合整数规划、求解代价升高，故本文改用分层优化。即先最小化购电费用，再在费用不变的前提下最小化充放电量，在不引入整数变量的同时排除同时充放电。求解得到全天购电费 **35126.95 元**，较不配置储能的 48052.05 元降低 **26.90%**。

针对问题二，负荷与光伏由确定值变为随机变量，购电计划需要在费用与缺电风险之间权衡。本文先用 **LightGBM** 对负荷与光伏做滚动预测，预测和场景构造均遵循**前向滚动原则**，只用决策时刻之前已观测的数据，不把未来信息带入模型。再由历史残差提取 30 个等概率场景刻画不确定性，建立**两阶段随机规划模型**：0:00 锁定计划购电量，日内由储能充放电、紧急购电与弃电补偿波动，并把求解窗口从单日扩展到 48 小时、给末端储能计价值，避免有限时域把储能放空。全年回测总购电费 **1335.79 万元**，紧急购电量仅占 **0.69%**，以较低的成本有效规避了供电不足的风险。

针对问题三，日内能不断获得更新的光伏预报，但更新带来的信息价值并不相同，需判断在哪些时刻调整计划最有利。本文把 0:00、6:00、12:00、18:00 的滚动预报纳入调度，建立**滚动时域模型**，每次更新只对尚未执行的时段重新优化。同时，任一决策时刻仅使用此前已观测数据和当时可获得的光伏预报，未来真实数据仅用于事后回测，避免前视偏差。比较四种策略后，采用 0:00 制定初始计划、6:00 与 12:00 各更新一次，全年总购电费 **1484.93 万元**，比仅 0:00 更新降低 **31.67 万元**，平衡了预报误差改善带来的收益与计划调整产生的额外成本。

针对问题四，外网电价进一步由固定电价扩展为实时波动电价，未来电价与净负荷均具有不确定性，且二者的误差存在相关性，不能简单当作独立量。因此，本文把外网电价也作为随机量，用 **K-medoids** 将净负荷与电价的预测误差各自标准化后联合聚类成**联合场景**，同时刻画两者的波动与相关性，并在此基础上重新求解问题二、问题三。波动电价下对应总购电费分别为 **1474.38 万元**与 **1568.42 万元**，较固定电价基准分别上升 10.38% 与 5.62%。

灵敏度分析表明，场景数与残差窗口在一定范围内变化时全年费用波动不超过 **2%**，结果较为稳定；各问求解均通过供需平衡与储能状态方程残差校验，储能电量全程处于约束区间，完整策略已按要求保存至 result1.xlsx 至 result4-3.xlsx。

关键字：微网 线性规划 两阶段随机规划 滚动时域优化 联合场景

一、 问题重述

1.1 问题背景

光伏等分布式电源在配电网侧的广泛接入，极大地推动了能源结构的转型。然而，其固有的间歇性与随机性也给局部电能消纳及系统的稳定运行带来了显著压力。为有效缓解上述并网问题，微网作为一种小型发用电系统得到了快速发展。作为集成分布式电源、储能设备与终端负荷的小型发用电系统，微网能够实现局部范围内的电能自平衡，有效提升了供电可靠性与新能源消纳率^[1]。

依托与外部电网的连接，并网式微网具备与主网进行实时电能交互的能力。在电力市场普遍实施分时电价的背景下，这种交互能力赋予了微网广阔的经济获利空间：系统在电价较低的时段为储能设备充电，在电价较高的时段由储能放电供给负载，从而减少高价时段的外网购电^[2]。因此，微网的电力调控需适应源荷功率的动态波动，并严格遵循储能设备的各项物理约束。核心目标是制定各时段的购电与充放电策略，在保障负载需求的前提下实现总购电成本的最小化。

1.2 问题提出

本题要求围绕微网与外网的电力调控策略，递进式地解决以下四个问题：

问题一：在每天的外网电价、小区负载维持固定，且给定光伏发电功率单日预测数据的理想条件下，要求制定 0:00 的日前计划购电策略。模型要求微网供电总量不可低于小区负载，储能设备在 0:00 与 24:00 的储电量保持一致。

问题二：在外网电价相同，但光伏发电功率和小区负载随时间产生真实波动的复杂情境下，微网需在每天 0:00 制定当天的计划购电策略，以应对不确定性带来的经济风险。调控规则为若实际供电低于负载，需向外网进行紧急购电，其电价为交易时刻电价的 5 倍；除紧急购电外，其余时段购电费用仍按计划购电量计算。

问题三：每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点的光伏发电功率预报，微网可依据更新后的预报调整购电策略，但需承担一定的调整成本：计划购电量高于调整购电量的部分按交易时刻电价的 50% 计费，调整购电量高于计划购电量的部分按交易时刻电价的 1.5 倍计费，总购电费用包括计划购电费、紧急购电费和调整购电相关费用。需要基于此机制建立综合考虑计划、调整与紧急购电的数学模型。

问题四：外网电价由固定转为实时波动。要求在动态波动电价环境下，重新建立并计算问题二与问题三中的优化模型。即在波动电价机制下，综合考量储能设备的套利空间、紧急购电成本等，给出适应高频电价波动的购电策略。

二、 问题分析

微网购电策略的核心是利用储能设备在不同时段之间搬移电量，在满足负载的前提下降低购电费用。四个问题由浅入深，逐步加入不确定性与时变因素，下面分别分析各问的思路与难点。

2.1 问题一的分析

问题一是在电价、负载与光伏预测均已知的条件下进行的单日调度，属于信息完全已知的确定性优化问题。调度的关键在于处理好时间与电量的动态平衡。每一时刻微网须满足功率平衡，同时储能电量受上下限与充放电功率的限制，且 0:00 与 24:00 的电量须相等。为了有效降低全天的购电费用，系统应当在电价较低的时段充电蓄能，在电价较高的时段放电供给负载。这一问是后续三个问题的基础。

2.2 问题二的分析

问题二引入光伏与负载随时间的实际波动，当计划供电低于负载时，须按交易时刻电价的 5 倍紧急购电。由于日前计划与实际需求存在偏差，若计划过于贴近负载下限，一旦供电不足将产生高额紧急购电费用；若计划留有余量，又会增加正常购电费用。因此，我们需利用历史数据了解光伏与负载的波动大小和误差分布，在制定计划时适当留出电量余量，以较小的成本抵消供电不足的风险。

2.3 问题三的分析

问题三在问题二的基础上，增加了每天 0:00、6:00、12:00、18:00 四个时刻的光伏预报更新，微网可根据最新预报调整购电计划，但调整会产生费用：计划购电高于调整购电的部分按电价的 50% 计费，调整购电高于计划购电的部分按电价的 1.5 倍计费。其难点在于是否调整、何时调整的权衡：不调整可能因预报不准而缺电，调整又需承担额外费用。基本思路是每当到达新的预报时刻，将此前已发生的时段固定下来，仅依据最新预报重新计算后续时段的购电计划，并比较调整带来的费用与不调整可能造成的缺电损失，选择总费用更小的方案。

2.4 问题四的分析

问题四将外网电价由固定变为实时波动。其难点在于电价起伏使低价充电、高价放电的机会增多，但储能充放电存在能量损耗，只有电价差大到足以覆盖损耗时才值得充放电，队伍计划沿用前几问的框架，把固定电价替换为附件 4 的时变电价后重新计算，得到波动电价下的购电策略。

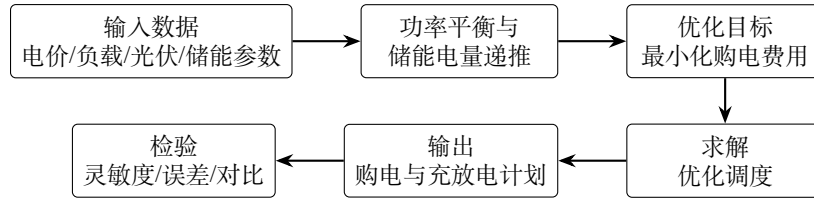


图 1 问题分析流程图

三、 模型假设

1. 假设微网内部电能传输无线路损耗，功率在时段切换时瞬时完成，且每个 10 分钟时段内电价、小区负载与光伏发电功率保持恒定。
依据：题目数据时间粒度为 10 分钟，该离散化是储能调度建模的必要前提。
2. 假设储能设备在同一时段内充放电不同时进行。
依据：物理上双向逆变器具备互斥逻辑，且经济优化模型也会自发避免因同时充放电带来的 90% 效率内耗。
3. 假设在所考察的全年周期内，储能设备的电池老化与容量衰减忽略不计，除充放电效率外无自放电及其它能量损耗，充放电效率恒定为 90%。
依据：单年度内电池容量衰减极微，将其降维处理为稳态线性系统能有效避免微观物理变化干扰宏观经济调整。
4. 假设微网不具备向外部电网反送电能的功能。在光伏发电出现盈余，储能设备处于满载状态的情况下，多余的光伏电能将直接作弃光处理。
依据：题目调控目标为节省购电费用，未提及上网电价，故不引入售电项。
5. 假设光伏功率的预测误差在周期内相对稳定，且忽略极端气象灾害可能引发的小概率异常突变。
依据：保证误差样本在统计上具有不变性，是问题二及问题三刻画不确定性、制定计划的基础。

四、 符号说明

本文主要符号及其含义见表 1。其中功率类变量（负载、光伏）以 kW 计，电量类变量由功率乘以时段时长 Δt 得到，单位为 kWh。

表 1 主要符号说明

符号	含义	单位
t	时段序号, $t = 1, 2, \dots, 144$	—
Δt	单时段时长, $\Delta t = 1/6$	h
p_t	时段 t 的外网电价	元/kWh
L_t	时段 t 的小区负载功率	kW
V_t	时段 t 的可用光伏发电功率	kW
q_t	时段 t 的微网计划购电量	kWh
E_t	时段 t 末的储能电量	kWh
e_t	时段 t 的紧急购电量	kWh
c_t, d_t	时段 t 的储能充电量、放电量	kWh
η	储能充放电效率	—

五、 模型准备

本节首先给出统一的单位口径、储能电量递推与能量平衡方程，作为后续各问建模的基础。问题一至问题四的模型建立与求解分别在各节展开。

5.1 单位口径统一

微网经济调度的基础是建立离散时间尺度的能量流转体系。题目给定的负载与光伏为瞬时功率，单位为 kW，而购电量与储能容量为电量，单位为 kWh，二者须通过时间步长换算。设调度时段长为 10 分钟，即

$$\Delta t = \frac{10}{60} = \frac{1}{6} \text{ h}, \quad (1)$$

全天共划分 144 个时段。设时段 t 的负载功率与光伏功率分别为 L_t 、 V_t ，则各时段的负载电量与光伏电量分别为 $L_t \Delta t$ 与 $V_t \Delta t$ ，单位均为 kWh。储能最大充放电功率为 5000kW，故单时段最大充放电量为

$$\bar{c} = \bar{d} = 5000 \times \frac{1}{6} \approx 833.33 \text{ kWh}. \quad (2)$$

若把 5000kW 直接当作 5000kWh 使用，会把储能充放电能力放大 6 倍。

5.2 储能电量递推关系

设储能设备的充放电效率均为 η 。充电时，送入储能设备的电量 c_t 中只有 ηc_t 转化为储能的净增加，其余为损耗；放电时，储能内部需消耗 d_t/η 才能对外提供电量 d_t 。于

是时段 t 末的储能电量 E_t 由时段初电量递推得到

$$E_t = E_{t-1} + \eta c_t - \frac{d_t}{\eta}. \quad (3)$$

储能电量须始终处于安全区间，记下限与上限分别为 $E_{\min}$ 、 $E_{\max}$ ，充放电量不超过单时段上限，即

$$E_{\min} \leq E_t \leq E_{\max}, \quad 0 \leq c_t \leq 833.33, \quad 0 \leq d_t \leq 833.33, \quad (4)$$

其中， $E_{\min} = 1200\text{kWh}$ 、 $E_{\max} = 10800\text{kWh}$ 。

5.3 能量方程

按基尔霍夫电流定律的思想，微网母线在任一时刻的能量输入必须等于能量输出与耗散之和。设 q_t 为计划购电量， e_t 为紧急购电量， u_t 为光伏实际利用量， w_t 为未被利用的盈余电量，则能量平衡方程为

$$q_t + e_t + u_t + d_t = L_t \Delta t + c_t + w_t. \quad (5)$$

等式左端为供给侧，由计划购电、紧急购电、光伏利用与储能放电构成；右端为消耗侧，由负载、储能充电与弃电构成。其中光伏利用量满足

$$0 \leq u_t \leq V_t \Delta t, \quad (6)$$

弃电量 w_t 吸收光伏与计划购电中无法被负载或储能消纳的盈余部分，所有变量均非负。问题一处于确定性环境，无紧急购电，即 $e_t = 0$ ；问题二、问题三中 e_t 按供电缺口确定。

5.4 充放电互斥简化

严格物理模型要求储能设备不能在同一时段既充电又放电，需引入 0-1 变量将模型变为混合整数规划。但本题中电价非负、充放电存在损耗、且允许弃电，同一时段同时充放电并无经济收益，最优解不会主动同时充放电。因此各问主模型保持为线性规划，仅在求解后校验是否出现同时充放电，若出现再补充 0-1 约束修正。

5.5 参数说明

模型中的主要参数取值见表 2。

表 2 模型主要参数取值

参数	含义	取值
Δt	调度时段时长	1/6h
η	储能充放电效率	0.9
$\bar{c}, \bar{d}$	单时段最大充/放电量	5000/6 $\approx$ 833.33kWh
$E_{\min}, E_{\max}$	储能电量上下限	1200, 10800kWh
E_0	储能初始电量	6000kWh
λ_{em}	紧急购电价格倍数	5
λ_{dn}	计划下调违约计费比例	0.5
λ_{up}	计划上调超调计费比例	1.5

六、 问题一的模型建立与求解

问题一处于电价、负载与光伏预测均已知的确定性环境，无需紧急购电，即 $e_t = 0$ ，且储能设备在 0:00 与 24:00 的电量相等。在模型准备建立的储能递推式 (3)与能量平衡式 (5)基础上，本节建立问题一的优化模型。

6.1 模型的建立

6.1.1 目标函数

微网调度的经济目标是最小化全天总购电费用

$$\min \sum_{t=1}^{144} p_t q_t. \quad (7)$$

6.1.2 约束条件

问题一的约束由三部分组成。

一是能量守恒，将 $e_t = 0$ 代入式 (5)，得

$$q_t + u_t + d_t = L_t \Delta t + c_t + w_t. \quad (8)$$

二是储能状态与物理边界，由式 (3)、(4)与式 (6)给出。

三是循环边界对齐，为保证多日连续调度，要求首末储能电量相等，即

$$E_0 = E_{144} = 6000. \quad (9)$$

6.1.3 完整优化模型

综合式 (7)–(9)，模型一为如下线性规划^[3]：

$$\begin{aligned}
& \min \sum_{t=1}^{144} p_t q_t \\
& \text{s.t. } q_t + u_t + d_t = L_t \Delta t + c_t + w_t, \quad t = 1, \dots, 144, \\
& E_t = E_{t-1} + \eta c_t - \frac{d_t}{\eta}, \quad t = 1, \dots, 144, \\
& E_0 = E_{144} = 6000, \quad E_{\min} \leq E_t \leq E_{\max}, \\
& 0 \leq c_t, d_t \leq 833.33, \quad 0 \leq u_t \leq V_t \Delta t, \\
& q_t, u_t, c_t, d_t, w_t \geq 0.
\end{aligned} \tag{10}$$

模型含 5×144 个连续决策变量，目标函数与全部约束均为决策变量的线性函数。

6.2 模型的求解

团队采用两阶段层次优化求解模型 (10)，并用线性规划求解器 HiGHS 完成计算。若仅以购电费用为目标，当多个解达到同一最低费用时，可能出现同一时段既充电又放电的问题，这类解虽然满足数学约束，却不符合储能设备的物理特性。

为在不引入 0–1 互斥变量的前提下保证解的物理合理性，我们计划分两阶段求解：第一阶段直接求解模型 (10)，得到最低购电费用；第二阶段在购电费用保持最低的前提下，将目标函数替换为最小化全天总充放电量 $\sum_t (c_t + d_t)$ 。由于充放电存在损耗，同一时段内的充电与放电行为相互抵消后并不能带来收益，只会增加总充放电量，因此在第二阶段目标下最优解不会同时充电和放电。这样既消除了冗余的充放电，又保持了线性规划求解的高效性。求解完成后，本文对供需平衡残差与储能状态方程残差进行了检验。

6.3 求解结果

模型的计算结果如表 3、表 4 所示。

表 3 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量	59482.70	全天购电费	35126.95 元		

表 4 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量	6000.00		24:00 储电量	6000.00	

6.4 结果分析

图2给出日内能源环境与购电响应。当日电价最低约 0.37 元/kWh、最高约 1.40 元/kWh，存在明显的峰谷价差，光伏全天发电量约 6247.96kWh。这种价格结构使储能可以在低价时段充电、在高峰时段放电，从而减少高价时段的外网购电。

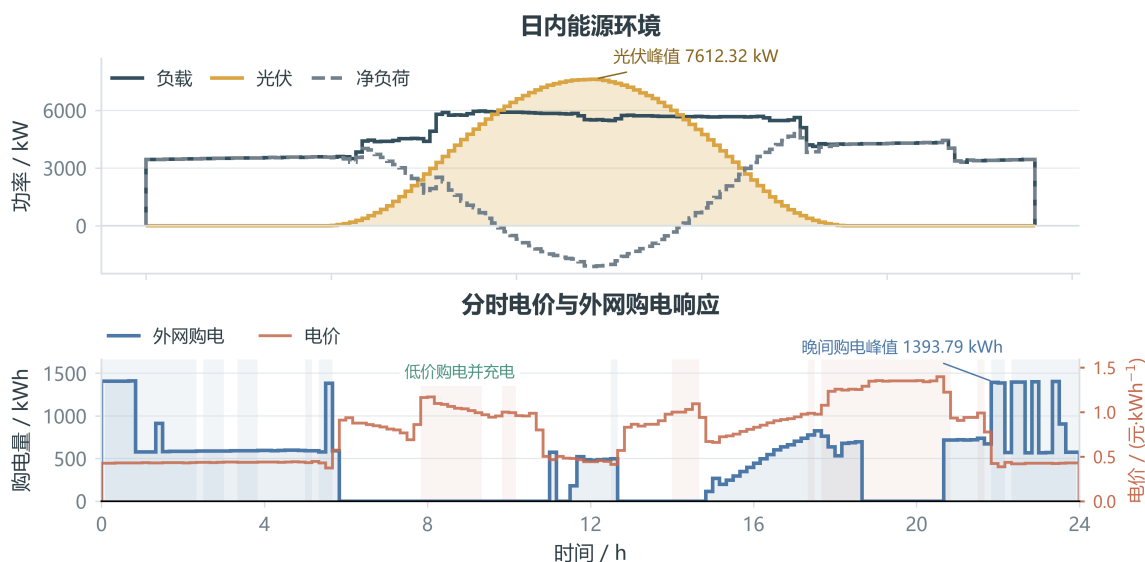


图 2 日内能源环境与购电响应

由表4，储能设备在 0:00–4:00 夜间充电 4500.00kWh，在 4:00–8:00 与 16:00–20:00 两个高峰时段分别放电 6365.84kWh 和 5780.13kWh，全天充电总量 20740.66kWh、放电总量 16799.94kWh，储能电量始终保持在 1200–10800kWh 区间内，无任何时段同时充放电，充放电与电量轨迹如图3所示。同时，表3中 12:00–12:10、16:00–16:10、18:00–18:10 三个高价时段的购电量明显高于其余时段。这说明储能将夜间低价电能转移到高峰时段使用，使微网的购电集中到光伏不足的高价时段，其余时段的负荷由储能放电与光伏共同承担。

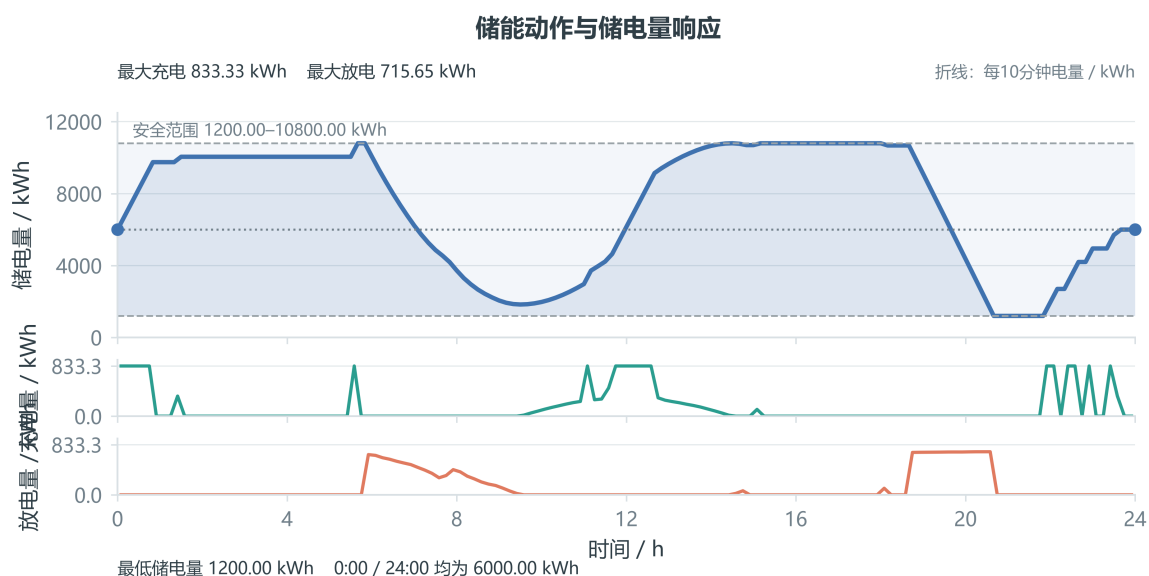


图3 储能充放电与电量轨迹

如图4所示,若微网不配置储能、仅靠光伏与购电满足负载,全天购电费用为48052.05元;采用本文模型后降至35126.95元,节省12925.10元,降幅达26.90%,显著提高了经济效益。

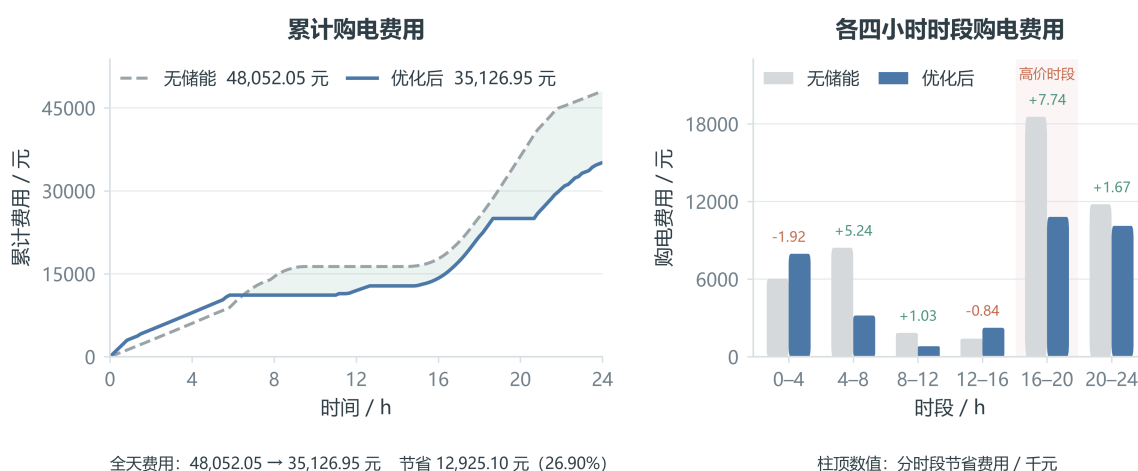


图4 经济性对比

上述购电与充放电计划均满足储能运行的约束: 首末储电量均为6000.00kWh, 储能电量全程保持在1200–10800kWh区间内,最大充放电量分别为833.33kWh与715.65kWh,未超过时段上限,无弃光、无同时充放电。供需平衡残差最大值为 2.27×10^{-13} kWh、储能状态方程残差最大值为 8.31×10^{-13} kWh,均小于 10^{-5} kWh的容差阈值,求解结果通过物理一致性检验。

七、 问题二的模型建立与求解

本节沿用模型准备与问题一的储能调度模型，建立问题二含补偿决策的两阶段随机规划模型。

7.1 模型的建立

7.1.1 场景生成

由于负载与光伏的实际值未知，本文采用场景法刻画其不确定性。先用 LightGBM^[4] 对负载和光伏功率做滚动预测，每天只用此前已经观测到的数据训练，得到当天的基准预测；再取最近 30 组相邻两日的样本外预测误差轨迹，分别叠加到当日与次日的基准预测上，构造 30 个等概率场景，即 $w_s = 1/30$ ，每个场景对应 48 小时（当日与次日）的实际负载与光伏，以保留日内与跨日的误差相关性。

7.1.2 两阶段随机规划模型

问题二采用两阶段随机规划^[5, 1]。每天 0:00 在 48 小时滚动窗口（当日与次日）内优化，但只执行并输出当日 144 个时段的计划购电量，次日仅作预览，第二天获得新信息后重新预测与优化。第一阶段确定对所有场景共享的计划购电量 q_t ；第二阶段在场景实现后，由储能充电量 $c_{s,t}$ 、放电量 $d_{s,t}$ 、紧急购电量 $e_{s,t}$ 与弃电量 $w_{s,t}$ 补偿波动。

对场景 s ，记其负载功率与光伏发电功率分别为 $L_{s,t}$ 、 $V_{s,t}$ ，净负荷电量为 $(L_{s,t} - V_{s,t})\Delta t$ 。各时段能量平衡要求计划购电、储能放电与紧急购电之和，等于净负荷、储能充电与弃电量之和，即

$$q_t + e_{s,t} + d_{s,t} = (L_{s,t} - V_{s,t})\Delta t + c_{s,t} + w_{s,t}. \quad (11)$$

由式 (11) 可知，紧急购电量 $e_{s,t}$ 即为该时段供电缺口，当计划购电与储能放电不足以覆盖净负荷时取正，否则为零。

储能电量递推沿用模型准备的式 (3)，对场景 s 为

$$E_{s,t} = E_{s,t-1} + \eta c_{s,t} - \frac{d_{s,t}}{\eta}, \quad (12)$$

并要求 $E_{\min} \leq E_{s,t} \leq E_{\max}$ 、 $0 \leq c_{s,t}, d_{s,t} \leq 833.33$ 。储能电量跨日连续传递，每日末电量即为次日初电量，即 $E_{d+1,0} = E_{d,24}$ 。

为避免 48 小时有限时域在窗口末端把储能放空，给窗口末端剩余储能赋予经济价值：每 1kWh 库存按放电效率乘以日内电价中位数计价，记为

$$\lambda_E = \eta \text{med}\{p_t\} = 0.9 \times 0.8074 \approx 0.7266 \text{ 元/kWh}. \quad (13)$$

目标为计划购电费与紧急购电费期望之和，再减去窗口末端剩余储能价值

$$\min \sum_{t=1}^T p_t q_t + \sum_{s \in \Omega} w_s \sum_{t=1}^T 5 p_t e_{s,t} - \lambda_E \sum_{s \in \Omega} w_s E_{s,T}, \quad (14)$$

其中，第一项为计划购电费，第二项为各场景紧急购电费的期望，第三项为 48 小时窗口末端的剩余储能价值。

综合式 (11)–(14)，问题二的完整模型为

$$\begin{aligned} \min & \sum_{t=1}^T p_t q_t + \sum_{s \in \Omega} w_s \sum_{t=1}^T 5 p_t e_{s,t} - \lambda_E \sum_{s \in \Omega} w_s E_{s,T} \\ \text{s.t.} & q_t + e_{s,t} + d_{s,t} = (L_{s,t} - V_{s,t})\Delta t + c_{s,t} + w_{s,t}, \quad \forall s, t = 1, \dots, T, \\ & E_{s,t} = E_{s,t-1} + \eta c_{s,t} - \frac{d_{s,t}}{\eta}, \quad \forall s, t = 1, \dots, T, \\ & E_{\min} \leq E_{s,t} \leq E_{\max}, \quad 0 \leq c_{s,t}, d_{s,t} \leq 833.33, \\ & q_t, c_{s,t}, d_{s,t}, e_{s,t}, w_{s,t} \geq 0. \end{aligned} \quad (15)$$

其中 $T = 288$ 为 48 小时滚动窗口的时段数， q_t 为所有场景共享的第一阶段决策，其余变量为场景相关的第二阶段变量；模型只执行前 144 个时段的购电与储能计划，次日 144 个时段仅作预览。

7.2 模型的求解

求解分为规划与执行两步。

在规划阶段，对每个计划窗口求解模型 (15)，得到日前计划购电量 q_t 。储能充放电互斥由分层优化实现。先以总购电费最小求解线性规划，再在费用不变的前提下最小化各场景总充放电量 $\sum_s \sum_t (c_{s,t} + d_{s,t})$ 。同时充放电不会降低购电费用，只会增加总充放电量，因此第二层目标会自然排除同时充放电。

在执行阶段，按自然日滚动执行，储能电量跨日连续传递。将计划购电量 q_t 固定，代入当日实际的负载与光伏，先最小化紧急购电费用，再最小化储能充放电量，得到当日的储能充放电、紧急购电与弃电量，供电缺口按 5 倍电价紧急购电。求解结果均通过了供需平衡残差与储能状态方程残差校验。

7.3 求解结果

本文对 2025 年 2 月 1 日至 12 月 31 日共 334 天做滚动求解。每天 0:00 按当天的基准预测与残差场景确定全天的计划购电量；当日实际数据确定后，按实际净负荷计算储能调度与紧急购电，计划购电量不再修改。四个指定日期的计划购电、储能充放电与紧急购电结果分别见表 5、表 6 与表 7。

表 5 四个指定日期的计划购电量及全天购电量、购电费

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量	全天购电费
2025.3.20	0	457.53	0	487.54	663.96	0	63082.99	39237.98
2025.6.21	0	0	0	126.99	0	0	40712.31	22442.71
2025.9.23	0	435.73	0	584.36	736.55	52.44	68579.44	43233.67
2025.12.21	0	893.78	0	852.24	686.49	0	95236.20	61279.69

注：表中购电量单位为 kWh，购电费单位为元。

计划购电量集中在 12:00、16:00 与 18:00 前后，10:00、14:00 与 20:00 购电为零。四个日期的全天购电量随季节负荷变化，12 月 21 日最高、6 月 21 日最低。

表 6 四个指定日期的储能充放电量

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00-4:00	3021.44	55.89	493.68	164.18	2170.85	30.31	2242.72	201.75
4:00-8:00	1752.21	4566.11	881.83	1739.99	1210.95	4947.10	2089.19	1642.69
8:00-12:00	4349.38	2777.38	0	0	4130.27	1896.62	4955.28	6755.81
12:00-16:00	4832.36	254.72	8841.84	0	5636.92	421.30	7730.28	2395.30
16:00-20:00	24.49	5647.56	960.58	5454.06	62.69	5716.51	293.74	5621.74
20:00-24:00	3189.76	3117.00	6120.65	2542.16	5941.31	2730.68	6878.00	2905.55
0:00 储电量	6744.70		2077.78		6794.01		7280.49	
24:00 储电量	3954.44		6646.06		6540.02		7358.72	

注：表中电量单位均为 kWh。

储能在夜间充电、在 4:00–8:00 与 16:00–20:00 两个高峰时段放电，6 月 21 日因光伏充足，储能充电集中在正午时段。

表 7 四个指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
15:30-16:50	370.75	无	0	14:50-15:10	136.29	无	0

注：表中购电量单位为 kWh。

全年汇总，计划购电量为 21070471.88kWh，紧急购电量为 146315.59kWh，后者仅占总购电量的 0.69%，富余电量为 2081891.36kWh；全年计划购电费为 1293.45 万元，紧急购电费为 42.33 万元，总购电费为 1335.79 万元。

7.4 结果分析

图 5 给出四个代表日预测净负荷与实际净负荷的对应关系，以及各日 P10–P90 经验区间的覆盖率。散点大致沿对角线分布，滚动预测没有系统偏差。由历史残差构造的

P10-P90 场景区间覆盖了实际波动的主要范围。全年负载预测的平均绝对误差、均方根误差分别为 127.87kW、172.92kW，光伏预测分别为 158.94kW、319.38kW，光伏的预测误差整体大于负载，与其波动性更强的特点一致。

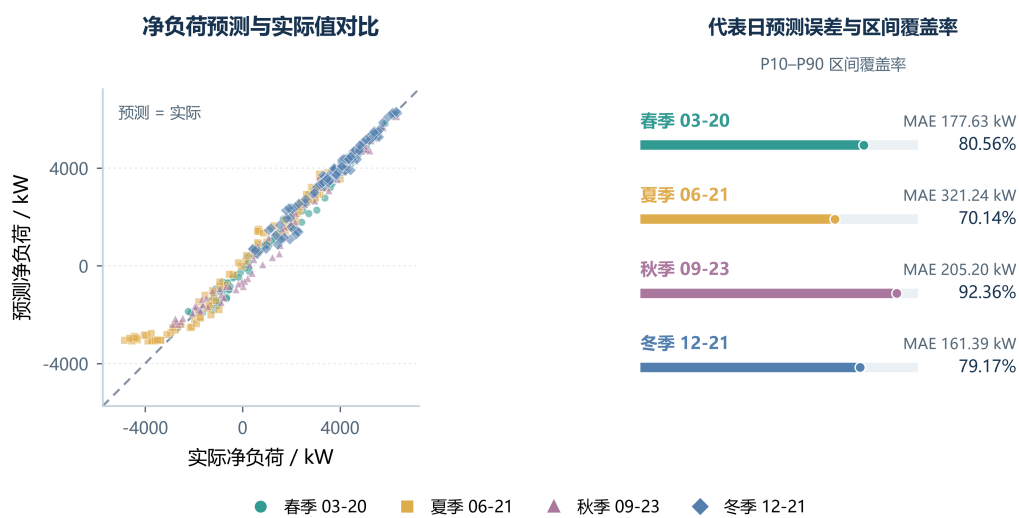


图 5 四个代表日的预测一致性及误差区间覆盖率

由图 6 可见，紧急购电主要集中在正午前后与午后两个时段。正午前后光伏预测偏高时，实际发电不足会留下缺口；午后光伏发电功率下降而负荷仍处高位，一旦净负荷高于计划就会触发紧急购电。结合表 7，四个指定日期中仅春分和秋分两天出现紧急购电，时段也落在上述两个区间内。

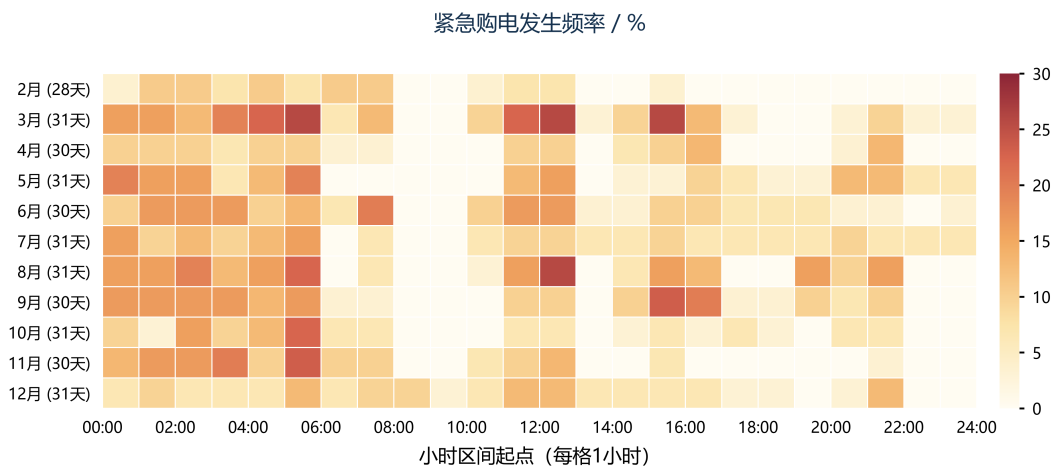


图 6 紧急购电风险的月度一小时分布

从全年各月的费用看（图 7），计划购电费在冬季和夏季较高，与负荷的季节变化一致；紧急购电费在 6 月、3 月、7 月最高，分别为 8.73 万元、5.81 万元、5.48 万元，其余月份更低。全年紧急购电量占总购电量不足 1%，对应的紧急购电费占总购电费约 3.2%。

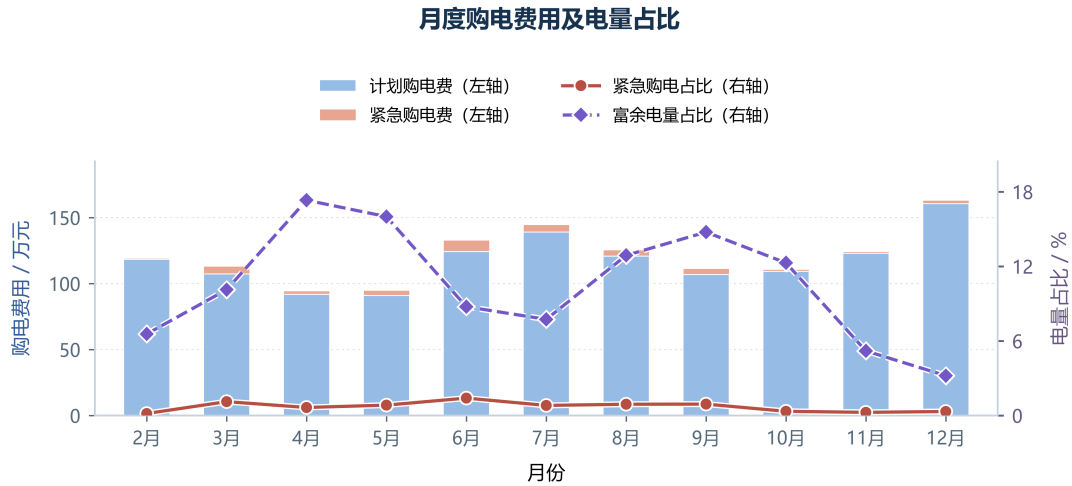


图7 月度购电费用与电量风险

为验证场景法与48小时滚动窗口的效果，本文模型与无储能点预测、24小时点预测、净负荷上浮10%或20%、7场景、15场景等策略做了全年回测对比，如图8所示。与这些基准策略相比，本文模型在减少紧急购电的同时没有增加富余电量，说明按历史残差构造的48小时场景能更细致地区分高、低风险时段。

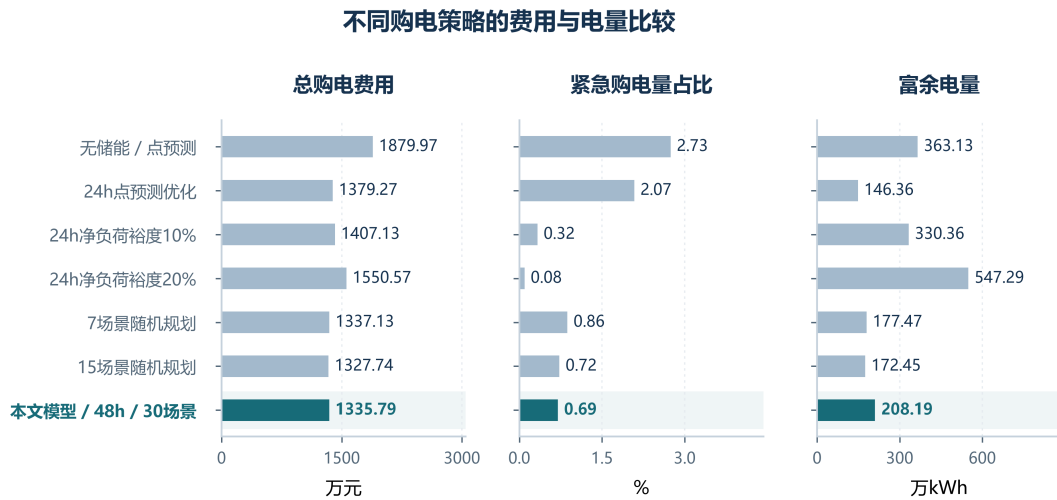


图8 不同购电策略的全年费用与风险对比

八、 问题三的模型建立与求解

问题三在问题二的基础上加入日内光伏预报更新，微网可在0:00、6:00、12:00、18:00调整已定的购电计划。调整后购电量高于计划购电量的部分按交易时刻电价的1.5倍计费，计划购电量高于调整后购电量的部分按交易时刻电价的50%计费。

8.1 模型的建立

8.1.1 计划调整的表达

设调整前的计划购电量为 $\bar{q}_t$ ，调整后的计划为 a_t 。把调高的部分记为 Δ_t^+ ，调低的部分记为 Δ_t^- ，两者都非负，则

$$a_t = \bar{q}_t + \Delta_t^+ - \Delta_t^-. \quad (16)$$

由于同一时段只会调高或调低， Δ_t^+ 与 Δ_t^- 不会同时为正。

8.1.2 初始计划

0:00 时还没有已定的计划，按问题二的模型求解得到全天初始计划 q_t 。与问题二一致，每天在 48 小时滚动窗口（当日与次日）内优化，只执行当日 144 个时段，储能电量跨日连续，窗口末端计入剩余储能价值 λ_E （定义见问题二式 (13)），目标为

$$\min \sum_{t=1}^T p_t q_t + \sum_{s \in \Omega} w_s \sum_{t=1}^T 5 p_t e_{s,t} - \lambda_E \sum_{s \in \Omega} w_s E_{s,T}. \quad (17)$$

约束与问题二式 (15) 相同。

8.1.3 调整费用

上调量 Δ_t^+ 对应的新增电量按交易时刻电价的 1.5 倍计费，产生额外费用 $1.5 p_t \Delta_t^+$ ；下调量 Δ_t^- 对应的削减电量按交易时刻电价的 50% 退款，相比原计划少支付 $0.5 p_t \Delta_t^-$ 。一次调整的净费用为

$$F_{\text{adj}} = \sum_{t \in T_{\text{rem}}} (1.5 p_t \Delta_t^+ - 0.5 p_t \Delta_t^-), \quad (18)$$

其， T_{rem} 为尚未执行的时段集合。同一时段不会同时出现上调和下调，故上式逐时段相加即可，无需引入整数变量。

8.1.4 更新时刻模型

每次更新对尚未执行的时段重新优化，目标为调整净费用与剩余时段各场景紧急购电费经验期望之和，并同样在 48 小时窗口末端计入剩余储能价值

$$\min \sum_{t \in T_{\text{rem}}} (1.5 p_t \Delta_t^+ - 0.5 p_t \Delta_t^-) + \sum_{s \in \Omega} w_s \sum_{t \in T_{\text{rem}}} 5 p_t e_{s,t} - \lambda_E \sum_{s \in \Omega} w_s E_{s,T}. \quad (19)$$

对已经发生的时段，计划不再改变，即 $\Delta_t^+ = \Delta_t^- = 0$ ；能量平衡与储能约束同问题二，仅将购电量替换为调整后的 a_t 。

8.2 模型的求解

问题三采用滚动时域方法求解^[6, 7]。0:00 求解初始计划，此后每个更新时刻获得新预报后，把已经执行的时段固定，仅对尚未执行的时段重新求解，每次调整只作用于未来时段。储能充放电互斥沿用问题二的做法：在目标函数中对充放电量加极小惩罚，使费用相同的最优解不会出现同时充放电。

附件 3 给出的光伏预报是未来 24 小时的整点预测。为得到 10 分钟间隔的预测，本文把发布时刻已观测到的实际光伏作为起点，在它与后续整点预测之间做线性插值，并保证插值结果非负。负荷采用前向滚动预测，场景用最近 28 个完整日的净负荷预测残差曲线经 k-medoids 缩减为 7 个场景，与问题二相同^[1]。

8.3 求解结果

按 0:00、6:00、12:00、18:00 四个更新时刻的不同组合，本文对 2025 年 2 月 1 日至 12 月 31 日共 334 天做全年滚动求解，四种策略的费用汇总见表 8。

表 8 不同预报更新策略的全年费用

策略	全年总费用/元	初始计划费/元	调整净费用/元	紧急购电费/元
仅 0:00	15,165,986.15	13,561,255.24	0.00	1,604,730.90
0:00+6:00	14,922,026.21	13,568,077.93	92,523.80	1,261,424.48
0:00+6:00+12:00	14,849,289.32	13,567,104.63	61,571.89	1,220,612.79
0:00+6:00+12:00+18:00	14,845,224.22	13,568,504.31	66,080.20	1,210,639.71

注：全年评价期为 2 月 1 日至 12 月 31 日共 334 天。

全年看，储能电量跨日连续，期初 SOC 为 8550kWh、全程保持在 1200–10800kWh 之间，各策略间无跨日连续残差。四个指定日期的紧急购电、购电与储能充放电结果分别见表 9、表 10 与表 11。

表 9 四个指定日期的紧急购电量

时间段	2025.3.20	2025.6.21	2025.9.23	2025.12.21
0:00-4:00	322.93	19.82	348.35	46.62
4:00-8:00	64.44	54.01	107.51	0.00
8:00-12:00	907.12	0.00	123.43	299.01
12:00-16:00	550.88	18.03	109.88	10.27
16:00-20:00	135.16	0.00	184.67	96.15
20:00-24:00	135.31	0.00	368.68	53.62

注：表中购电量单位为 kWh，受篇幅限制，表中按 4 小时时段汇总展示紧急购电量；各指定日期具体的紧急购电起止时刻及对应购电量详见支撑材料 result3.xlsx 中“紧急购电量”工作表。

表 10 四个指定日期的购电量及全天购电量、购电费

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量	全天购电费
2025.3.20	0	531.63	0	509.72	751.10	0	63168.75	49128.22
2025.6.21	0	0	0	181.75	503.01	0	45701.04	26169.31
2025.9.23	0	381.77	0	454.33	809.16	0	68758.05	49030.23
2025.12.21	0	918.11	0	832.86	725.79	0	100377.90	67149.33

注：购电量单位为 kWh、购电费单位为元；全天购电费为计划购电费、调整净费用与紧急购电费之和。

表 11 四个指定日期的储能充放电量及储电量

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00-4:00	1824.39	0	610.73	0	2077.88	0	1682.29	0
4:00-8:00	833.33	5707.33	881.99	2171.26	833.33	5997.68	1656.13	913.74
8:00-12:00	4670.35	2932.67	3807.64	0	4960.69	2643.80	5148.81	8262.71
12:00-16:00	6025.41	278.80	6837.04	37.35	6159.59	394.41	8858.18	2891.93
16:00-20:00	318.13	4947.91	74.32	5208.57	117.46	4865.62	144.42	4959.85
20:00-24:00	4509.29	3740.23	8074.13	3495.10	7920.45	3872.74	8111.02	3786.17
0:00 储电量	8408.05		2288.85		8179.91		8535.94	
24:00 储电量	5207.60		8421.36		8293.21		8449.58	

注：表中电量单位均为 kWh。

8.4 结果分析

由（图 9）可知，光伏预报更新的价值首先体现在预测误差的下降上。6:00 更新使剩余时段光伏预测的平均绝对误差由 259.80kW 降至 158.52kW、下降 38.99%，12:00 更新再降至 75.25kW、下降 48.26%，18:00 更新仅下降 1.11%（图 9）。

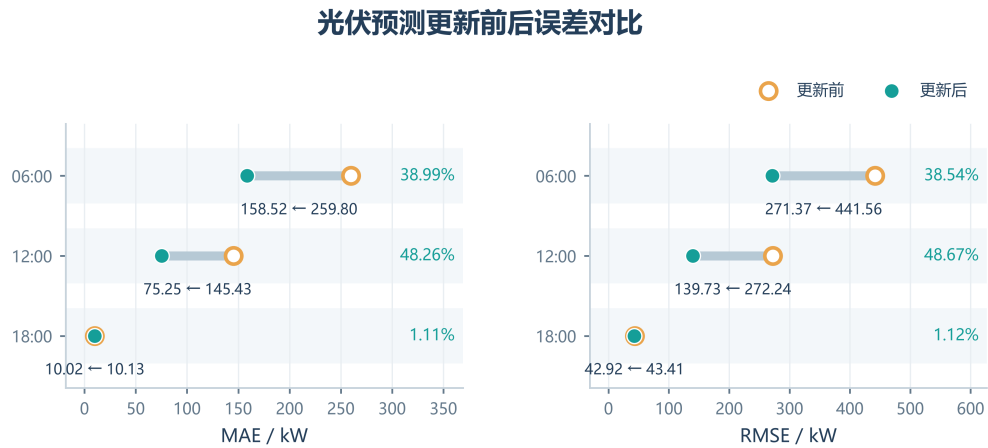


图 9 各更新时刻更新前后的光伏预测误差

图 10给出各更新时刻的边际信息价值：6:00 更新约 24.40 万元，12:00 更新约 7.27 万元，18:00 更新仅约 0.41 万元。18:00 更新的收益远小于前两个时刻，与表 8 中 0:00+6:00+12:00+18:00

策略仅比 0:00+6:00+12:00 策略低 0.41 万元一致，即预测误差的下降转化为购电费用的节省。

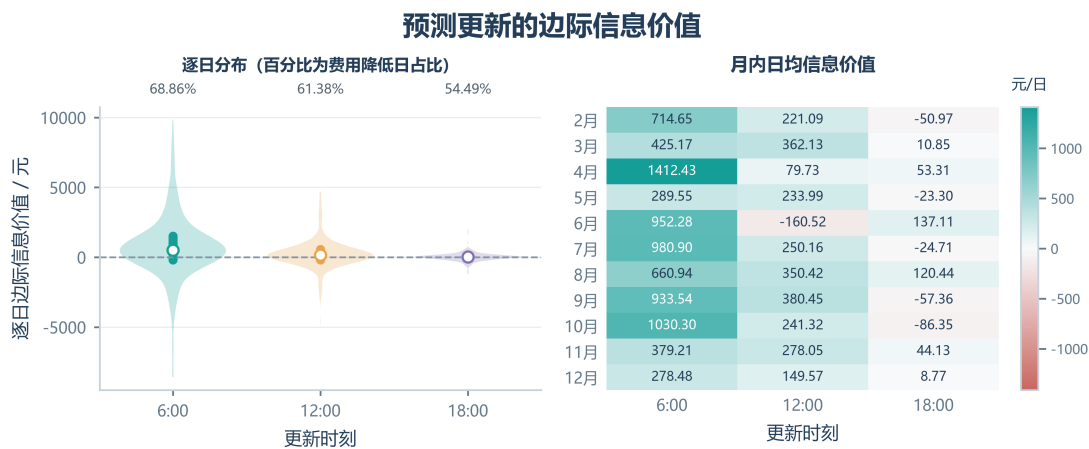


图 10 各预报更新时刻的信息价值

推荐策略下购电计划的调整分布见图 11。调整集中在 6:00 与 12:00 更新之后：当新预报显示光伏发电功率低于此前预期时，模型增加购电计划；反之则减少购电计划。由于调整需要承担额外费用，模型只有在调整带来的购电费用下降超过调整费用时才调整，因此调整量整体不大。

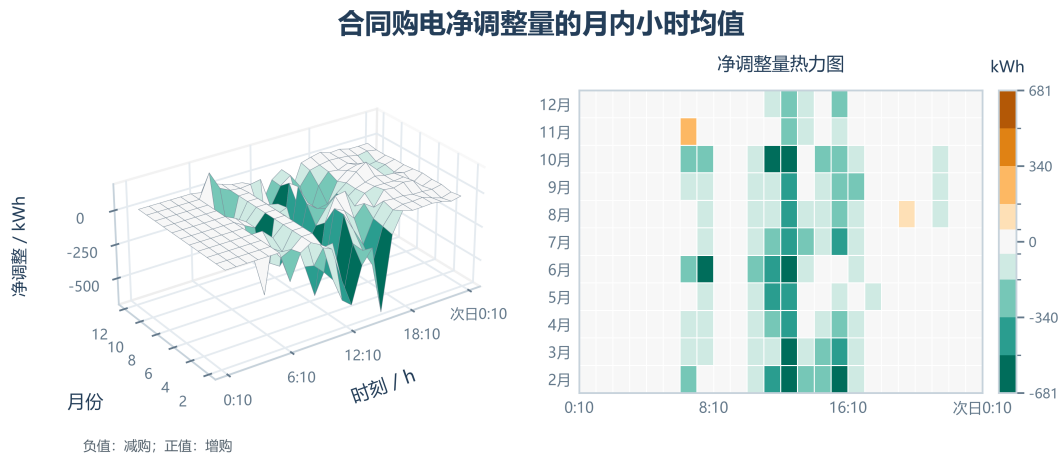


图 11 购电计划的调整分布

从全年费用看，6:00 与 12:00 的更新分别节省约 24.40 万元与 7.27 万元，18:00 的更新仅节省约 0.41 万元、光伏预测误差改善不足 1.1%。因此本文采用 0:00 制定初始计划、6:00 与 12:00 进行滚动修正的策略。该策略全年最终购电量为 21670349.58kWh，紧急购电量为 321726.44kWh，后者占总购电量的 1.46%，富余电量为 2735586.00kWh；全年计划购电费为 1362.87 万元，紧急购电费为 122.06 万元，总购电费为 1484.93 万元。

8.5 与问题二费用差异的说明

section 与问题二费用差异的说明

需要指出的是，问题三的全年总费用高于问题二，这并不意味着日内预报更新降低了调度效果。两问虽然采用相同的储能调度框架，但其决策信息集和场景构造方式并不完全一致。问题二基于历史数据对负荷和光伏进行滚动预测，并利用历史预测残差构造场景；问题三则进一步引入附件 3 在各更新时刻实际发布的光伏预报，并基于新的预测信息重新构造滚动场景，同时还需计入购电计划调整产生的相关费用。因此，两问的绝对费用并不是在完全相同的预测基准下得到的，不宜仅根据问题二与问题三的总费用大小判断预报更新是否有效。

为评价日内预报更新的实际价值，本文在问题三内部采用相同的信息结构和模型参数，比较不同更新策略。结果表明，仅在 0:00 制定计划时全年总费用为 1516.60 万元，加入 6:00 和 12:00 两次预报更新后降至 1484.93 万元，减少 31.67 万元。由此可见，在相同的问题三基准下，日内滚动更新能够有效利用新增预报信息，降低预测偏差带来的紧急购电和调度成本。

九、 问题四的模型建立与求解

问题四把外网电价由固定值改为随日期 d 与时段 t 同时变化的实时电价 $p_{d,t}$ 。0:00 制定购电计划时，未来电价与净负荷一样未知，需要同时处理两者的不确定性；日内调整也按实际电价结算。

9.1 电价与净负荷预测

0:00 时只能用此前观测到的数据。电价基准预测取前 1、7、14 日同一时刻电价的加权平均

$$\hat{p}_{d,t}^{(0)} = \sum_{k \in \{1,7,14\}} \alpha_k p_{d-k,t}, \quad (20)$$

权重 α_k 非负且和为 1，由历史数据拟合得到。6:00、12:00、18:00 更新时，先算出当天已观测价格与基准预测的偏差，再把这个偏差加到未来时段的预测上。负荷预测沿用问题二的方法，光伏功率由附件 3 的整点预报插值得到。

9.2 场景生成

对历史每一天计算净负荷与电价的预测误差，两者各自除以标准差后合并，用 k-medoids 聚成 $K = 7$ 类，类中心作为该场景的误差曲线，场景概率 π_s 取各类样本占比。净负荷与电价放在同一个场景里刻画，保留它们在同一天内的相关性。

9.3 问题四-2：波动电价下的两阶段随机规划

问题四-2 在问题二的两阶段结构上改动两处：场景由净负荷残差扩展为净负荷与电价的联合场景，计划购电量由固定电价计价改为期望电价计价。第一阶段确定所有场景共享的计划购电量 q_t ，第二阶段由储能充放电、紧急购电与弃电补偿波动，能量平衡与储能递推同问题二式 (11)、(12)。期望电价为 $\bar{p}_t = \sum_s \pi_s p_{s,t}$ ，紧急购电仍按交易时刻电价的 5 倍计价，末端储能价值按 $\lambda_E = \eta \text{med}\{p_{d,t}\} \approx 0.7737$ 元/kWh 计算。目标函数为

$$\min \sum_{t=1}^T \bar{p}_t q_t + \sum_{s \in \Omega} \pi_s \sum_{t=1}^T 5 p_{s,t} e_{s,t} - \lambda_E \sum_{s \in \Omega} \pi_s E_{s,T}. \quad (21)$$

约束与问题二式 (15) 一致，其中 $T = 288$ 。计划购电量只在决策阶段按期望电价确定，实际结算按当日真实电价计费。

9.4 问题四-3：波动电价下的滚动调度

问题四-3 对应问题三的滚动调度，电价换成场景电价。0:00 先按问题四-2 的模型确定初始计划，6:00、12:00、18:00 再对尚未执行的时段重新优化；调整关系同问题三式 (16)，增购按实际电价的 1.5 倍计费、减购按 50% 退回。更新时刻的目标为

$$\min \sum_{t \in T_{\text{rem}}} \left(1.5 \bar{p}_t \Delta_t^+ - 0.5 \bar{p}_t \Delta_t^- \right) + \sum_{s \in \Omega} \pi_s \sum_{t \in T_{\text{rem}}} 5 p_{s,t} e_{s,t} - \lambda_E \sum_{s \in \Omega} \pi_s E_{s,T}, \quad (22)$$

其中 T_{rem} 为尚未执行的时段集合，约束同问题四-2。

9.5 模型的求解

求解方法与问题二、问题三相同：先以费用最小求解线性规划，再在费用不变的前提下最小化充放电量，以排除同时充放电；问题四-3 按滚动时域执行，储能电量跨日连续。求解结果均通过供需平衡与储能状态方程残差校验。

9.6 求解结果

对 2025 年 2 月 1 日至 12 月 31 日共 334 天做全年回测，波动电价下问题四-2 的全期总购电费为 1474.38 万元，其中计划购电费 1378.33 万元、紧急购电费 96.05 万元；全年紧急购电量为 305738.77 kWh，富余电量为 2374027.31 kWh，评价期初、期末储电量分别为 7800.00 kWh 与 7105.25 kWh。四个指定日期的计划购电、储能充放电与紧急购电结果分别见表 12、表 13 与表 14。

表 12 波动电价下问题四-2 四个指定日期的购电量及全天购电量、购电费

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量	全天购电费
2025.3.20	0	0	0	681.66	0	722.38	63368.25	40989.04
2025.6.21	0	0	0	71.37	458.51	0	41369.88	20924.96
2025.9.23	0	380.25	0	592.64	0	0	68224.68	45062.34
2025.12.21	0	976.66	0	868.57	733.48	0	98261.76	74900.97

注：购电量单位为 kWh，购电费为计划购电费、单位为元。

表 13 波动电价下问题四-2 四个指定日期的储能充放电量及储电量

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00-4:00	4751.74	300.54	704.79	604.58	2384.49	211.85	0	317.31
4:00-8:00	2281.23	5908.29	2123.40	1719.95	1240.17	6163.09	0	1387.44
8:00-12:00	3107.48	2777.38	0	0	4217.93	1896.62	4790.93	5318.49
12:00-16:00	6944.57	254.72	7896.14	0	6800.47	403.62	7190.55	2656.93
16:00-20:00	645.74	6878.32	1064.84	4772.49	260.81	7286.75	437.34	5473.59
20:00-24:00	1112.49	1724.27	3245.26	2485.90	5076.43	1451.93	9036.83	2900.32
0:00 储电量	4854.99		1237.44		7124.67		10050.00	
24:00 储电量	1987.80		4120.73		5758.19		9300.00	

注：表中电量单位均为 kWh。

表 14 波动电价下问题四-2 四个指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
0:20-0:30	9.08	无	0	21:40-21:50	30.91	无	0
4:50-5:10	9.65						
5:30-5:50	604.35						
7:00-7:20	255.72						
7:30-7:40	36.60						

注：表中购电量单位为 kWh。

问题四-3 按仅 0:00、0:00+6:00、0:00+6:00+12:00、0:00+6:00+12:00+18:00 四种更新策略滚动求解，全年总费用见表 15。加入 6:00 与 12:00 更新后费用逐次下降，再加入 18:00 更新费用反而略有回升，因此采用 0:00+6:00+12:00 策略，其全年总费用为 1568.42 万元。该策略下四个指定日期的结果见表 16–18。

表 15 波动电价下问题四-3 不同更新策略的全年费用

策略	全年总费用/元	初始计划费/元	调整净费用/元	紧急购电费/元
仅 0:00	16051853.58	14288199.50	0.00	1763654.07
0:00+6:00	15772847.18	14285943.70	119094.58	1367808.90
0:00+6:00+12:00	15684219.65	14283214.23	100661.43	1300343.98
0:00+6:00+12:00+18:00	15690541.30	14281527.16	103743.69	1305270.45

注：全年评价期为 2 月 1 日至 12 月 31 日共 334 天。

表 16 波动电价下问题四-3 四个指定日期的购电量及全天购电量、购电费

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量	全天购电费
2025.3.20	0	527.56	0	561.13	0	743.06	63566.31	50124.14
2025.6.21	0	0	0	222.08	484.68	0	41488.96	20842.07
2025.9.23	0	376.39	0	546.94	0	0	68492.40	50097.86
2025.12.21	0	923.30	0	859.80	736.40	0	99380.79	78601.22

注：购电量为最终购电量、单位为 kWh；全天购电费为计划购电费、调整净费用与紧急购电费之和、单位为元。

表 17 波动电价下问题四-3 四个指定日期的储能充放电量及储电量

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00-4:00	4856.09	0	645.57	663.29	3462.88	0	868.72	0
4:00-8:00	1666.67	6235.65	2403.82	1947.10	833.33	6320.67	834.01	1628.90
8:00-12:00	4926.60	2273.23	5921.13	0	5627.08	2319.33	6015.34	7331.22
12:00-16:00	5919.76	314.29	5383.25	527.20	5361.72	260.93	7557.13	2845.59
16:00-20:00	80.21	7232.07	43.64	5837.07	140.82	7371.38	179.89	5688.13
20:00-24:00	884.17	1488.53	3501.53	2843.85	4983.54	1493.93	8932.79	2961.11
0:00 储电量	4929.52		1355.97		6933.41		10018.16	
24:00 储电量	1936.59		4333.36		5561.57		9239.51	

注：表中电量单位均为 kWh。

表 18 波动电价下问题四-3 四个指定日期的紧急购电量

时间段	2025.3.20	2025.6.21	2025.9.23	2025.12.21
0:00-4:00	346.29	8.17	98.66	32.82
4:00-8:00	134.71	19.68	47.75	0.00
8:00-12:00	761.22	0.00	36.60	406.52
12:00-16:00	559.53	0.00	285.70	20.02
16:00-20:00	83.46	0.00	172.18	91.45
20:00-24:00	128.25	0.00	326.76	15.99

注：表中购电量单位为 kWh。

9.7 结果分析

全年各小时实际电价的分布见图 12。电价在日内和日间都有明显起伏，峰谷价差较大，储能因此可以在低价时段充电、高价时段放电来降低成本^[2]，日前计划对电价预测的依赖也更强。

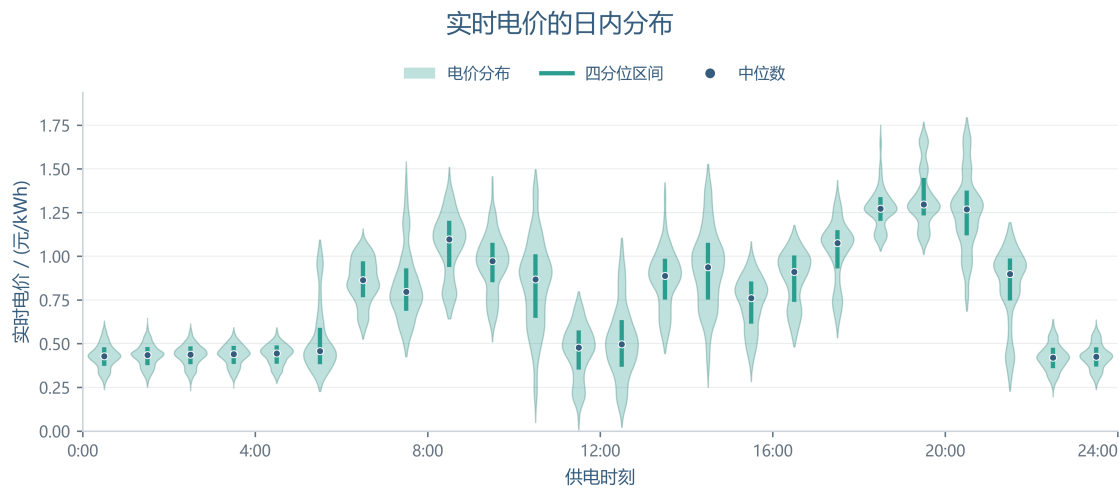


图 12 全年各小时实际电价的分布

储能的充放电方向始终跟随电价。图 13按电价高低分档统计了净充放电功率，低价档以充电为主，高价档以放电为主，全程没有同时充放电。

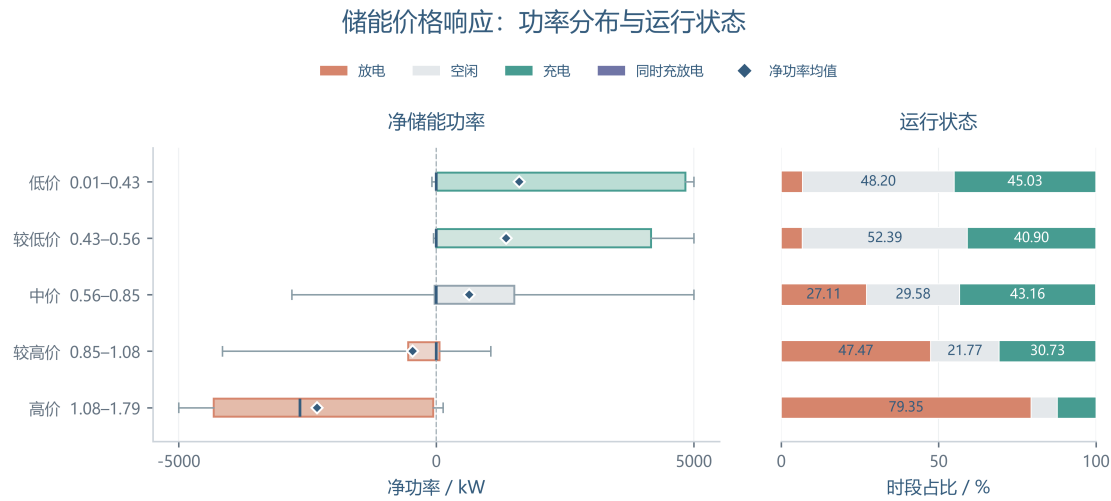


图 13 储能净充放电功率随电价分档的分布

由图 14，对未执行合同的修订主要集中在 6:00 与 12:00 两次更新之后，18:00 更新几乎没有修订，与表 15中 18:00 更新费用回升一致。

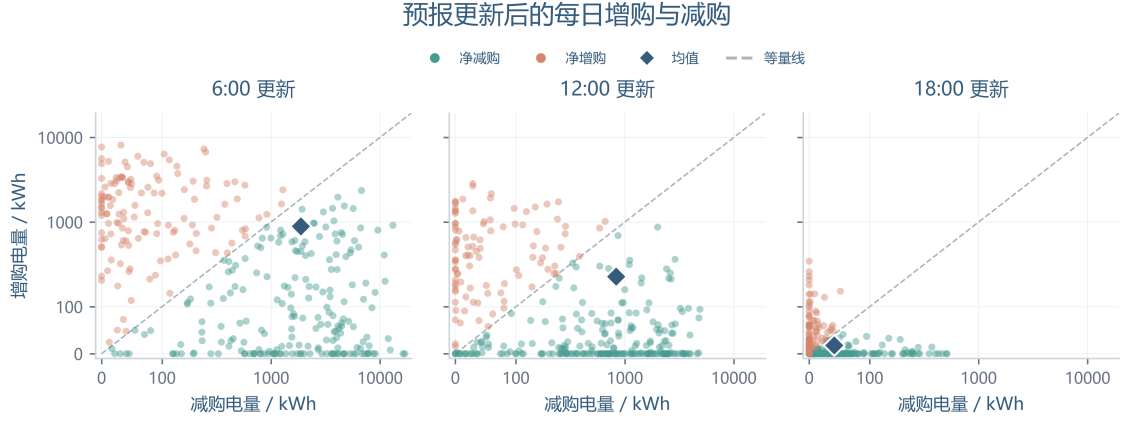


图 14 各更新时刻的合同修订量分布

与固定电价相比，波动电价抬高了购电成本：问题四-2 的总购电费比问题二高 10.38%，紧急购电量由 146315.59 kWh 增至 305738.77 kWh；问题四-3 采用 0:00+6:00+12:00 策略的总购电费比问题三高 5.62%。费用上升一方面来自高价时段购电成本更高，另一方面来自电价预测误差带来的紧急购电与调整费用。即便如此，6:00 与 12:00 两次更新仍把总费用由 1605.19 万元降到 1568.42 万元。

十、 敏感性分析与模型鲁棒性检验

为检验所建立的 SAA-MPC 滚动购电模型对关键参数的敏感程度，本文以 $K = 7$ 、 $H = 28$ 、 $L = 1$ 、 $V = 1$ 作为基准方案进行全年回测。其中 K 为代表场景数， H 为历史窗口长度， L 为跨日前视天数， V 为末端储能价值系数。

为量化评估参数扰动对模型输出的影响，定义总费用相对变化率 δ_J 与紧急购电量相对变化率 δ_E 为：

$$\delta_J = \frac{J - J_0}{J_0} \times 100\% \quad (23)$$

$$\delta_E = \frac{E_{em} - E_{em,0}}{E_{em,0}} \times 100\% \quad (24)$$

基准方案下的全年总费用 $J_0 = 1.4844989 \times 10^7$ 元，紧急购电量 $E_{em,0} = 3.222996 \times 10^5$ kWh。各参数扰动下的回测结果如表 19 所示。

表 19 问题三关键参数敏感性分析结果

试验组	参数设置	总费用/万元	$\delta_J/\%$	紧急购电量/万 kWh	$\delta_E/\%$	期末 SOC/kWh	日末接近上界天数
场景数	$K = 5$	1502.04	+1.182	38.87	+20.592	8533.80	0
基准	$K = 7, H = 28, L = 1, V = 1$	1484.50	0	32.23	0	8583.08	0
场景数	$K = 9$	1476.12	-0.565	28.78	-10.714	8545.81	0
历史窗	$H = 14$	1475.87	-0.581	28.55	-11.416	8574.63	0
历史窗	$H = 42$	1486.13	+0.110	36.76	+14.046	8517.44	0
前视长度	$L = 0$	1489.77	+0.355	31.98	-0.761	10800.00	307
前视长度	$L = 2$	1484.43	-0.005	32.21	-0.077	8583.08	0
终端价值	$V = 0$	1484.50	0	32.23	0	8583.08	0
终端价值	$V = 0.5$	1484.50	0	32.23	0	8583.08	0
终端价值	$V = 1.5$	1484.50	0	32.23	0	8583.08	0

基于上述回测数据，可以得出以下结论：

- 场景数与历史窗口对经济性的影响：改变代表场景数或历史样本窗口长度，会引起紧急购电量出现 10% 到 20% 的明显波动，但全年总费用的变动幅度始终控制在 1.2% 以内。这表明调整这两个参数主要影响系统对极端预测偏差的应对策略，而不会从根本上改变整体的购电成本水平。综合考虑求解效率与模型精度，基准方案的设定最为合理。
- 跨日前视长度的必要性：跨日前视长度对储能状态的连续性起决定作用。若取消跨日前视（令 $L = 0$ ），模型由于缺乏对次日运行状态的统筹，会导致全年 334 天中有 307 天的日末储电量异常贴近容量上限。引入单日前视（ $L = 1$ ）即可有效消除这一短视现象，保证储能状态跨日平滑过渡。进一步将前视长度延长至两天时，总费用的下降微乎其微，说明单日前视已能充分满足调度需求。
- 终端价值系数的独立性：在已确立跨日前视的滚动架构下，系统能够获取充足的未来运行信息。因此，对终端价值系数进行任意扰动，全年的各项经济指标与储能状态均保持完全一致。这说明该系数不会主导长期的经济决策，模型对该参数不具备依赖性。

十一、 模型的评价与推广

11.1 模型的优点

1. 全文四个问题统一使用线性规划与随机线性规划框架。问题一先建立确定性的储能调度模型，问题二加入场景法刻画负荷与光伏的不确定性，问题三引入日内预报更新与购电调整，问题四再把电价作为随机量并入场景，每一步都只在前一问的基础上增加变量与约束，没有引入非线性项。储能充放电互斥由分层优化处理，避免了 0-1 整数变量带来的求解开销。
2. 不确定性的处理遵循前向滚动原则。负荷、光伏与电价的预测只用决策时刻之前已

经观测到的数据，不把未来信息带入预测；场景由历史残差构造，净负荷与电价放在同一个场景里，保留了它们在同一天内的相关性。48 小时滚动窗口配合末端储能价值，避免了有限时域在窗口末端把储能放空，也使相邻两天的计划能够平滑衔接。

3. 问题三、四的滚动调整与题目规定的违约电价机制对应，增购按交易时刻电价的 1.5 倍计费、减购按 50% 退款。回测结果表明，加入 6:00 与 12:00 两次更新后，固定电价下全年费用由 1516.60 万元降至 1484.93 万元，波动电价下由 1605.19 万元降至 1568.42 万元，日内更新带来稳定的费用下降。

11.2 模型的缺点

1. 模型以 10 分钟为最小调度单元，时段内负荷、光伏与电价视为恒定，没有刻画时段内部更短时间尺度的波动，这一粒度由题目附件数据本身决定。
2. 负荷、光伏与电价的预测采用加权平均和线性插值等简单方法，窗口末端储能价值 λ_E 用历史电价中位数近似。

11.3 模型的推广

本文的滚动调度与联合场景框架可以直接用于含多种分布式电源、可中断负荷或需求响应资源的微网日前一日内调度，只需在能量平衡方程中增减相应项。若把联合场景扩展到负荷、光伏与电价三者同时波动，并用分布鲁棒优化刻画场景概率的不确定性，本模型也可以用于电价波动更频繁的电力现货市场。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【算法讨论，语言润色，代码调试等】，详细使用情况见支撑材料。

参考文献

- [1] Liu H, Tang W. Dynamic battery degradation-informed optimal microgrid scheduling: A prediction-aware two-stage stochastic programming approach with spatiotemporal correlation scenario generation[J]. Journal of Energy Storage, 2026, 165: 122231. DOI: 10.1016/j.est.2026.122231.
- [2] Prakash A, Bruce A, MacGill I. The scheduling role of future pricing information in electricity markets with rising deployments of energy storage: An Australian National Electricity Market case study[J]. Energy Economics, 2025, 142: 108191. DOI: 10.1016/j.eneco.2025.108191.

- [3] 《运筹学》教材编写组. 运筹学：本科版（第 5 版）[M]. 北京：清华大学出版社，2022.
- [4] Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu T Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]// Advances in Neural Information Processing Systems 30. Long Beach: Curran Associates, 2017: 3146–3154.
- [5] 陈铭宏天, 耿江海, 赵雨泽, 等. 基于两阶段随机优化的电氢耦合微电网周运行策略 [J]. 中国电力, 2025, 58(5): 82–90. DOI: 10.11930/j.issn.1004-9649.202408035.
- [6] Saha S, Elliott M, Huda S, Abawajy J, Jiang F, Oo A M T, Sun Y, Mao T. Model predictive control-based energy management for microgrids in weak utility grid environments[J]. International Journal of Electrical Power & Energy Systems, 2026, 181: 112066. DOI: 10.1016/j.ijepes.2026.112066.
- [7] Rawlings J B, Mayne D Q, Diehl M M. Model Predictive Control: Theory, Computation, and Design[M]. 2nd ed. Madison, WI: Nob Hill Publishing, 2017.

附录

支撑材料说明

本文随论文一并提交支撑材料，主要包括各问题的求解程序、计算结果及作图文件，并根据要求，打包在 ZIP 文件中：

问题一支撑材料包括 `problem1.py`、`result1.xlsx` 及相应结果图，用于复现确定性条件下的线性规划购电与储能调度结果；

问题二包括 `problem2.py`、`result2.xlsx` 及相关图形，其中结果文件记录全年计划购电、储能充放电和紧急购电结果；

问题三包括 `problem3.py`、`result3.xlsx`、灵敏度分析程序 `run_sensitivity.py` 以及不同参数组合下的计算结果，用于复现滚动预测、场景缩减、购电计划调整和灵敏度分析过程；

问题四包括 `problem4.py`、`result4-2.xlsx`、`result4-3.xlsx` 及相关图形，分别对应实时电价条件下问题二和问题三模型的重新计算结果。

此外，支撑材料中保存了论文主要图形的可编辑或高分辨率版本，以及模型计算过程中输出的结果数据，便于对正文中的表格、图形及主要结论进行复核。AI 工具的具体使用情况另附于 AI 工具使用详情.pdf 中。

1.1 源程序代码

由于篇幅限制，此处未展示所有代码只保留核心建模过程，所有代码详见附件，以下为本论文建模使用到的核心源代码：

1.1.1 问题一求解代码

Listing 1 问题一求解程序

```
1
2 """问题一求解程序。
3 读取附件1并完成两阶段线性规划求解，输出题目要求的结果。
4 时间标签按区间结束时刻处理。
5 运行方式：python problem1_core.py --input 附件1.xlsx
6 """
7 from __future__ import annotations
8 import argparse
9 import csv
10 from dataclasses import dataclass
11 from datetime import datetime, time, timedelta
12 from pathlib import Path
13 from typing import Optional
14 import numpy as np
15 import openpyxl
```

```

16 from scipy.optimize import linprog
17 from scipy.sparse import csr_matrix, lil_matrix
18
19 PERIODS = 144
20 DT_MINUTES = 10
21 DT_HOURS = 1 / 6
22
23 @dataclass(frozen=True)
24 class ModelParameters:
25     capacity_min: float = 1200.0
26     capacity_max: float = 10800.0
27     initial_energy: float = 6000.0
28     max_power: float = 5000.0
29     charge_efficiency: float = 0.90
30     discharge_efficiency: float = 0.90
31
32     @property
33     def max_energy_per_period(self) -> float:
34         """将最大充放电功率换算为单个10分钟时段的电量上限。"""
35         return self.max_power * DT_HOURS
36
37 @dataclass(frozen=True)
38 class InputData:
39     timestamps: list[object]
40     interval_labels: list[str]
41     price: np.ndarray
42     load_energy: np.ndarray
43     pv_energy: np.ndarray
44
45 @dataclass(frozen=True)
46 class Solution:
47     grid: np.ndarray
48     charge: np.ndarray
49     discharge: np.ndarray
50     pv_used: np.ndarray
51     energy: np.ndarray
52     total_grid: float
53     total_cost: float
54     baseline_cost: float
55     primary_cost: float
56     secondary_used: bool
57     max_balance_error: float
58     max_state_error: float
59     primary_throughput: float
60     final_throughput: float
61     simultaneous_periods: int
62     curtailed_energy: float
63

```

```

64 def timestamp_to_end_minute(value: object) -> int:
65     """将附件1中的时间标签统一换算为自0:00起的累计分钟数。"""
66     if isinstance(value, datetime):
67         return value.hour * 60 + value.minute
68
69     if isinstance(value, time):
70         return value.hour * 60 + value.minute
71
72     if isinstance(value, timedelta):
73         return int(round(value.total_seconds() / 60.0))
74
75     # 兼容Excel将时间存储为一天内小数的情况。
76     if isinstance(value, (int, float)) and not isinstance(value, bool):
77         minute = int(round(float(value) * 24 * 60))
78         if minute == 0:
79             return 0
80         return minute
81
82     text = str(value).strip().replace(":", ".").replace(" ", "")
83
84     if text in {"0:00+1", "00:00+1", "24:00", "24:00:00"}:
85         return 1440
86
87     if "+1" in text:
88         base = text.replace("+1", "")
89         parts = base.split(":")
90         if len(parts) < 2:
91             raise ValueError(f"无法解析时间标签: {value!r}")
92         return 1440 + int(parts[0]) * 60 + int(parts[1])
93
94     parts = text.split(":")
95     if len(parts) < 2:
96         raise ValueError(f"无法解析时间标签: {value!r}")
97
98     return int(parts[0]) * 60 + int(parts[1])
99
100 def format_minute(minute: int) -> str:
101     """将累计分钟数转换为结果表使用的时间格式。"""
102     if minute == 1440:
103         return "24:00"
104     if minute > 1440:
105         minute_next = minute - 1440
106         return f"{minute_next // 60}:{minute_next % 60:02d}+1"
107     return f"{minute // 60}:{minute % 60:02d}"
108
109 def build_physical_interval_labels(timestamps: list[object]) -> list[str]:
110     """检查时间轴并生成全天144个连续10分钟区间。"""
111     end_minutes = [timestamp_to_end_minute(v) for v in timestamps]

```

```

112     expected = list(range(DT_MINUTES, 24 * 60 + DT_MINUTES, DT_MINUTES))
113
114     if len(end_minutes) != PERIODS:
115         raise ValueError(f"附件1时间点应为{PERIODS}个，实际为{len(end_minutes)}个。")
116
117     for idx, (actual, wanted) in enumerate(zip(end_minutes, expected), start=1):
118         if actual != wanted:
119             raise ValueError(
120                 "附件1时间轴不符合A解释下的连续10分钟结束时刻： "
121                 f"第{idx}个时间点实际为{format_minute(actual)}, "
122                 f"期望为{format_minute(wanted)}。 "
123             )
124
125     labels: list[str] = []
126     for end_minute in end_minutes:
127         start_minute = end_minute - DT_MINUTES
128         labels.append(f"{format_minute(start_minute)}-{format_minute(end_minute)}")
129     return labels
130
131 def read_input(path: Path) -> InputData:
132     if not path.exists():
133         raise FileNotFoundError(f"找不到附件1: {path}")
134
135     workbook = openpyxl.load_workbook(path, data_only=True, read_only=True)
136     sheet = workbook.active
137     rows = list(sheet.iter_rows(min_row=2, values_only=True))
138     workbook.close()
139
140     if len(rows) != PERIODS:
141         raise ValueError(f"附件1应有{PERIODS}个时段，实际读取到{len(rows)}个。")
142
143     timestamps = [row[0] for row in rows]
144     interval_labels = build_physical_interval_labels(timestamps)
145
146     try:
147         matrix = np.asarray([row[1], row[2], row[3]] for row in rows, dtype=float)
148     except (TypeError, ValueError) as exc:
149         raise ValueError("附件1的电价、负载或光伏列存在非数值内容。") from exc
150
151     if not np.all(np.isfinite(matrix)):
152         raise ValueError("附件1的电价、负载或光伏列存在空值、NaN或无穷值。")
153     if np.any(matrix < 0):
154         raise ValueError("附件1的电价、负载和光伏预测不应出现负数。")
155
156     return InputData(
157         timestamps=timestamps,
158         interval_labels=interval_labels,
159         price=matrix[:, 0],

```

```

160     load_energy=matrix[:, 1] * DT_HOURS,
161     pv_energy=matrix[:, 2] * DT_HOURS,
162 )
163
164 def solve_problem(data: InputData, params: ModelParameters) -> Solution:
165     t_count = len(data.price)
166     n_variables = 5 * t_count + 1
167
168     # 决策变量在向量x中的分段位置: G, C, D, PV_use, E
169     grid = slice(0, t_count)
170     charge = slice(t_count, 2 * t_count)
171     discharge = slice(2 * t_count, 3 * t_count)
172     pv_used = slice(3 * t_count, 4 * t_count)
173     energy_start = 4 * t_count
174
175     # 等式约束由供需平衡和SOC状态递推两部分组成
176     a_eq = lil_matrix((2 * t_count, n_variables), dtype=float)
177     b_eq = np.zeros(2 * t_count, dtype=float)
178
179     for t in range(t_count):
180         #  $G_t + PV_{use\_t} + D_t = L_t + C_t$ 
181         a_eq[t, grid.start + t] = 1.0
182         a_eq[t, charge.start + t] = -1.0
183         a_eq[t, discharge.start + t] = 1.0
184         a_eq[t, pv_used.start + t] = 1.0
185         b_eq[t] = data.load_energy[t]
186
187         #  $E_{t+1} = E_t + \eta_c C_t - D_t / \eta_d$ 
188         row = t_count + t
189         a_eq[row, energy_start + t] = -1.0
190         a_eq[row, energy_start + t + 1] = 1.0
191         a_eq[row, charge.start + t] = -params.charge_efficiency
192         a_eq[row, discharge.start + t] = 1.0 / params.discharge_efficiency
193
194     # 设置各决策变量的取值范围
195     bounds: list[tuple[Optional[float], Optional[float]]] = []
196     bounds.extend([(0.0, None)] * t_count) #  $G_t \geq 0$ , 只允许购电
197     bounds.extend([(0.0, params.max_energy_per_period)] * t_count) #  $C_t$ 
198     bounds.extend([(0.0, params.max_energy_per_period)] * t_count) #  $D_t$ 
199     bounds.extend([(0.0, float(v)) for v in data.pv_energy]) #  $PV_{use\_t}$ 
200
201     # 固定日初、日末SOC, 中间时段满足容量约束
202     bounds.append((params.initial_energy, params.initial_energy))
203     bounds.extend([(params.capacity_min, params.capacity_max)] * (t_count - 1))
204     bounds.append((params.initial_energy, params.initial_energy))
205
206     # 第一阶段: 最小化购电费用
207     cost_vector = np.zeros(n_variables, dtype=float)

```

```

208     cost_vector[grid] = data.price
209
210     primary = linprog(
211         cost_vector,
212         A_eq=a_eq.tocsr(),
213         b_eq=b_eq,
214         bounds=bounds,
215         method="highs",
216     )
217     if not primary.success:
218         raise RuntimeError(f"第一阶段购电费用最小化失败: {primary.message}")
219
220     # 第二阶段: 保持最低购电费用, 并进一步最小化总充放电量
221     tolerance = max(1e-7, abs(primary.fun) * 1e-10)
222
223     throughput_vector = np.zeros(n_variables, dtype=float)
224     throughput_vector[charge] = 1.0
225     throughput_vector[discharge] = 1.0
226
227     a_ub = csr_matrix(cost_vector.reshape(1, -1))
228     b_ub = np.asarray([primary.fun + tolerance], dtype=float)
229
230     secondary = linprog(
231         throughput_vector,
232         A_ub=a_ub,
233         b_ub=b_ub,
234         A_eq=a_eq.tocsr(),
235         b_eq=b_eq,
236         bounds=bounds,
237         method="highs",
238     )
239
240     result = secondary if secondary.success else primary
241     x = np.where(np.abs(result.x) < 1e-8, 0.0, result.x)
242
243     grid_values = x[grid]
244     charge_values = x[charge]
245     discharge_values = x[discharge]
246     pv_used_values = x[pv_used]
247     energy_values = x[energy_start : energy_start + t_count + 1]
248
249     # 检查供需平衡和SOC递推的数值残差
250     balance_residual = (
251         grid_values
252         + pv_used_values
253         + discharge_values
254         - data.load_energy
255         - charge_values

```

```

256 )
257 state_residual = (
258     energy_values[1:]
259     - energy_values[:-1]
260     - params.charge_efficiency * charge_values
261     + discharge_values / params.discharge_efficiency
262 )
263
264 max_balance_error = float(np.max(np.abs(balance_residual)))
265 max_state_error = float(np.max(np.abs(state_residual)))
266
267 if max(max_balance_error, max_state_error) > 1e-5:
268     raise RuntimeError(
269         "求解结果未通过数值校验: "
270         f"最大供需平衡误差={max_balance_error:.3e}, "
271         f"最大SOC状态方程误差={max_state_error:.3e}。"
272     )
273
274 # 计算无储能情况下的购电费用基准
275 baseline_grid = np.maximum(data.load_energy - data.pv_energy, 0.0)
276
277 return Solution(
278     grid=grid_values,
279     charge=charge_values,
280     discharge=discharge_values,
281     pv_used=pv_used_values,
282     energy=energy_values,
283     total_grid=float(grid_values.sum()),
284     total_cost=float(data.price @ grid_values),
285     baseline_cost=float(data.price @ baseline_grid),
286     primary_cost=float(primary.fun),
287     secondary_used=bool(secondary.success),
288     max_balance_error=max_balance_error,
289     max_state_error=max_state_error,
290     primary_throughput=float(primary.x[charge].sum() + primary.x[discharge].sum()),
291     final_throughput=float(charge_values.sum() + discharge_values.sum()),
292     simultaneous_periods=int(np.count_nonzero((charge_values > 1e-6) & (discharge_values >
293     1e-6))),
294     curtailed_energy=float((data.pv_energy - pv_used_values).sum()),
295 )
296
297 def show_and_save(data, solution, output_dir, show_all=False):
298     s = solution
299     output_dir.mkdir(parents=True, exist_ok=True)
300     selected = {f"{h}:00-{h}:10" for h in (10, 12, 14, 16, 18, 20)}
301     print("表1: 指定时段购电量 (kWh) ")
302     for label, amount in zip(data.interval_labels, s.grid):

```

```

303     if label in selected:
304         print(f"{label}: {amount:.4f}")
305     print(f"全天购电量: {s.total_grid:.4f} kWh; 全天购电费: {s.total_cost:.4f} 元")
306     print("表2: 分段充电量、放电量 (kWh) ")
307     cb, db = s.charge.reshape(6, 24).sum(1), s.discharge.reshape(6, 24).sum(1)
308     for i in range(6):
309         print(f"{4*i}:00-{4*i+4}:00: {cb[i]:.4f}, {db[i]:.4f}")
310     print(f"0:00电量: {s.energy[0]:.4f}; 24:00电量: {s.energy[-1]:.4f} kWh")
311     with (output_dir / "计划购电量.csv").open("w", encoding="utf-8-sig", newline="") as f:
312         w = csv.writer(f); w.writerow(["时段", "购电量_kWh"])
313         w.writerows(zip(data.interval_labels, s.grid))
314         w.writerow(["全天购电量_kWh", s.total_grid]); w.writerow(["全天购电费_元", s.total_cost])
315     with (output_dir / "充放电量.csv").open("w", encoding="utf-8-sig", newline="") as f:
316         w = csv.writer(f); w.writerow(["时段", "充电量_kWh", "放电量_kWh"])
317         w.writerows((f"{4*i}:00-{4*i+4}:00", cb[i], db[i]) for i in range(6))
318         w.writerow(["0:00储电量_kWh", s.energy[0]]); w.writerow(["24:00储电量_kWh", s.energy
319         [-1]])
320     print("结果目录: ", output_dir.resolve())
321
322
323 def main():
324     parser = argparse.ArgumentParser(description="问题一附录核心程序 (含结果输出) ")
325     parser.add_argument("--input", type=Path, default=Path("附件1.xlsx"))
326     parser.add_argument("--output-dir", type=Path,
327                         default=Path(__file__).resolve().parent / "问题一_题目要求结果")
328     args = parser.parse_args()
329     data = read_input(args.input)
330     solution = solve_problem(data, ModelParameters())
331     show_and_save(data, solution, args.output_dir)
332
333
334 if __name__ == "__main__":
335     main()

```

1.1.2 问题二求解代码

Listing 2 问题二求解程序

```

1 """问题二求解程序。
2 采用LightGBM进行滚动预测，并基于历史预测残差构造48小时场景，
3 随后求解随机线性规划并完成固定购电计划下的实际调度。
4 可通过--end指定计算截止日期；默认计算至2025年12月31日。
5 """
6 from __future__ import annotations
7 import argparse, csv, math, json, warnings
8 from dataclasses import dataclass

```

```

9 from datetime import datetime, date, timedelta
10 from pathlib import Path
11 import numpy as np
12 import openpyxl
13 from scipy.optimize import linprog
14 from scipy.sparse import csr_matrix, lil_matrix
15 from lightgbm import LGBMRegressor
16 warnings.filterwarnings('ignore', message='X does not have valid feature names, but
    LGBMRegressor was fitted with feature names')
17 PERIODS = 144
18 DT_HOURS = 1 / 6
19 EPS_ZERO = 1e-08
20 EMERGENCY_THRESHOLD = 1e-06
21 from types import SimpleNamespace
22 YearData = ForecastRecord = DayPlan = ActualDispatch = DayResult = SimpleNamespace
23
24 @dataclass(frozen=True)
25 class ModelParameters:
26     capacity_min: float = 1200.0
27     capacity_max: float = 10800.0
28     initial_energy: float = 6000.0
29     max_power: float = 5000.0
30     charge_efficiency: float = 0.9
31     discharge_efficiency: float = 0.9
32     emergency_multiplier: float = 5.0
33     terminal_value_scale: float = 1.0
34
35     @property
36     def max_energy_per_period(self) -> float:
37         return self.max_power * DT_HOURS
38
39 @dataclass(frozen=True)
40 class ForecastConfig:
41     min_history_days: int = 7
42     residual_warmup_start_day: int = 14
43     scenario_days: int = 30
44     retrain_every: int = 1
45     n_estimators: int = 180
46     learning_rate: float = 0.05
47     num_leaves: int = 31
48     min_child_samples: int = 20
49     random_state: int = 2026
50
51 def format_clock_from_minutes(minutes: int, use_next_day_label: bool=True) -> str:
52     if minutes == 1440:
53         return '0:00+1' if use_next_day_label else '24:00'
54     minutes %= 1440
55     h, m = divmod(minutes, 60)

```

```

56     return f'{h}:{m:02d}'
57
58 def physical_interval_label(slot: int, use_next_day_label: bool=True) -> str:
59     start = slot * 10
60     end = (slot + 1) * 10
61     return f'{format_clock_from_minutes(start, use_next_day_label)}--{format_clock_from_minutes(
62         end, use_next_day_label)}'
63
64 def physical_interval_labels() -> list[str]:
65     return [physical_interval_label(t) for t in range(PERIODS)]
66
67 def parse_date_like(value: object) -> date:
68     if isinstance(value, datetime):
69         return value.date()
70     if isinstance(value, date):
71         return value
72     text = str(value).strip()
73     for fmt in ('%Y-%m-%d', '%Y/%m/%d', '%Y.%m.%d'):
74         try:
75             return datetime.strptime(text, fmt).date()
76         except ValueError:
77             pass
78     raise ValueError(f'无法识别日期: {value!r}')
79
80 def read_daily_price(path: Path) -> tuple[list[object], np.ndarray]:
81     if not path.exists():
82         raise FileNotFoundError(f'找不到附件1: {path}')
83     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
84     ws = wb.active
85     rows = list(ws.iter_rows(min_row=2, values_only=True))
86     wb.close()
87     if len(rows) != PERIODS:
88         raise ValueError(f'附件1应有 {PERIODS} 个时段, 实际 {len(rows)} 个。')
89     labels = [row[0] for row in rows]
90     price = np.asarray([row[1] for row in rows], dtype=float)
91     if not np.all(np.isfinite(price)) or np.any(price < 0):
92         raise ValueError('附件1电价存在空值、非数值或负数。')
93     return (labels, price)
94
95 def read_actual_year(path: Path) -> tuple[list[date], list[object], np.ndarray, np.ndarray]:
96     if not path.exists():
97         raise FileNotFoundError(f'找不到附件2: {path}')
98     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
99     required = ['小区负载', '光伏发电实际功率']
100     for name in required:
101         if name not in wb.sheetnames:
102             wb.close()
103             raise ValueError(f'附件2缺少工作表: {name}')

```

```

103 ws_load = wb['小区负载']
104 ws_pv = wb['光伏发电实际功率']
105 load_all = list(ws_load.iter_rows(values_only=True))
106 pv_all = list(ws_pv.iter_rows(values_only=True))
107 wb.close()
108 if not load_all or not pv_all:
109     raise ValueError('附件2工作表为空。')
110 if len(load_all[0]) != PERIODS + 1 or len(pv_all[0]) != PERIODS + 1:
111     raise ValueError('附件2每张表应为1列日期 + 144列10分钟数据。')
112 time_labels_load = list(load_all[0][1:])
113 time_labels_pv = list(pv_all[0][1:])
114 if [str(x) for x in time_labels_load] != [str(x) for x in time_labels_pv]:
115     raise ValueError('附件2负载与光伏工作表的时间列不一致。')
116 dates_load = [parse_date_like(row[0]) for row in load_all[1:]]
117 dates_pv = [parse_date_like(row[0]) for row in pv_all[1:]]
118 load_rows = [list(row[1:]) for row in load_all[1:]]
119 pv_rows = [list(row[1:]) for row in pv_all[1:]]
120 if dates_load != dates_pv:
121     raise ValueError('附件2负载与光伏工作表日期不一致。')
122 load_kw = np.asarray(load_rows, dtype=float)
123 pv_kw = np.asarray(pv_rows, dtype=float)
124 if load_kw.shape[1] != PERIODS or pv_kw.shape[1] != PERIODS:
125     raise ValueError('附件2数据列数不是144。')
126 if not (np.all(np.isfinite(load_kw)) and np.all(np.isfinite(pv_kw))):
127     raise ValueError('附件2存在空值或非数值。')
128 if np.any(load_kw < 0) or np.any(pv_kw < 0):
129     raise ValueError('附件2负载或光伏存在负数。')
130 return (dates_load, time_labels_load, load_kw, pv_kw)
131
132 def load_year_data(price_path: Path, actual_path: Path) -> YearData:
133     price_labels, price = read_daily_price(price_path)
134     dates, actual_labels, load_kw, pv_kw = read_actual_year(actual_path)
135     if len(price_labels) != len(actual_labels):
136         raise ValueError('附件1与附件2的日内时段数量不一致。')
137     if len(dates) < 365:
138         warnings.warn(f'附件2只有 {len(dates)} 天，代码仍会按实际天数运行。')
139     return YearData(dates=dates, time_labels=actual_labels, price=price, load_kw=load_kw, pv_kw
140                     =pv_kw)
141
142 def build_feature_tensor(series_kw: np.ndarray, dates: list[date]) -> np.ndarray:
143     n_days, n_periods = series_kw.shape
144     if n_periods != PERIODS:
145         raise ValueError('series_kw第二维必须是144。')
146     n_features = 13
147     X = np.full((n_days, PERIODS, n_features), np.nan, dtype=float)
148     slot = np.arange(PERIODS, dtype=float)
149     midpoint_hour = (slot * 10.0 + 5.0) / 60.0
150     tod_angle = 2.0 * np.pi * midpoint_hour / 24.0

```

```

150     for d in range(7, n_days):
151         dt = dates[d]
152         doy = dt.timetuple().tm_yday
153         doy_angle = 2.0 * np.pi * doy / 365.0
154         dow = dt.weekday()
155         hist3 = series_kw[d - 3:d]
156         hist7 = series_kw[d - 7:d]
157         X[d, :, 0] = slot
158         X[d, :, 1] = np.sin(tod_angle)
159         X[d, :, 2] = np.cos(tod_angle)
160         X[d, :, 3] = float(dow)
161         X[d, :, 4] = 1.0 if dow >= 5 else 0.0
162         X[d, :, 5] = math.sin(doy_angle)
163         X[d, :, 6] = math.cos(doy_angle)
164         X[d, :, 7] = series_kw[d - 1]
165         X[d, :, 8] = series_kw[d - 7]
166         X[d, :, 9] = np.mean(hist3, axis=0)
167         X[d, :, 10] = np.mean(hist7, axis=0)
168         X[d, :, 11] = np.std(hist7, axis=0)
169         X[d, :, 12] = float(dt.month)
170     return X
171
172 class RollingForecaster:
173
174     def __init__(self, load_kw: np.ndarray, pv_kw: np.ndarray, dates: list[date], cfg:
ForecastConfig) -> None:
175         self.load_kw = load_kw
176         self.pv_kw = pv_kw
177         self.dates = dates
178         self.cfg = cfg
179         self.load_X = build_feature_tensor(load_kw, dates)
180         self.pv_X = build_feature_tensor(pv_kw, dates)
181         self.load_model: LGBMRegressor | None = None
182         self.pv_model: LGBMRegressor | None = None
183         self.last_train_day: int | None = None
184
185     def _new_model(self) -> LGBMRegressor:
186         if LGBMRegressor is None:
187             raise ImportError('未安装 lightgbm。请在当前Python环境中执行 python -m pip install
lightgbm。')
188         return LGBMRegressor(objective='regression_l1', n_estimators=self.cfg.n_estimators,
learning_rate=self.cfg.learning_rate, num_leaves=self.cfg.num_leaves, min_child_samples=
self.cfg.min_child_samples, subsample=0.9, colsample_bytree=0.9, reg_lambda=0.1,
random_state=self.cfg.random_state, n_jobs=-1, verbosity=-1)
189
190     def _fit(self, target_day: int) -> None:
191         start_day = 7
192         if target_day - start_day < self.cfg.min_history_days:

```

```

193         raise ValueError(f'预测 {self.dates[target_day]} 时历史训练天数不足; 需要至少 {self.
cfg.min_history_days} 天。')
194     train_days = np.arange(start_day, target_day)
195     X_load = self.load_X[train_days].reshape(-1, self.load_X.shape[-1])
196     y_load = self.load_kw[train_days].reshape(-1)
197     X_pv = self.pv_X[train_days].reshape(-1, self.pv_X.shape[-1])
198     y_pv = self.pv_kw[train_days].reshape(-1)
199     self.load_model = self._new_model()
200     self.pv_model = self._new_model()
201     self.load_model.fit(X_load, y_load)
202     self.pv_model.fit(X_pv, y_pv)
203     self.last_train_day = target_day
204
205     def predict(self, target_day: int) -> tuple[np.ndarray, np.ndarray]:
206         if target_day < 14:
207             raise ValueError('滚动LightGBM预测默认从1月15日以后使用。')
208         need_retrain = self.load_model is None or self.pv_model is None or self.last_train_day
is None or (target_day - self.last_train_day >= self.cfg.retrain_every)
209         if need_retrain:
210             self._fit(target_day)
211         assert self.load_model is not None and self.pv_model is not None
212         pred_load = self.load_model.predict(self.load_X[target_day], validate_features=False)
213         pred_pv = self.pv_model.predict(self.pv_X[target_day], validate_features=False)
214         pred_load = np.clip(np.asarray(pred_load, dtype=float), 0.0, None)
215         pred_pv = np.clip(np.asarray(pred_pv, dtype=float), 0.0, None)
216         return (pred_load, pred_pv)
217
218     @staticmethod
219     def _recursive_next_features(series_kw: np.ndarray, dates: list[date], target_day: int,
current_prediction: np.ndarray) -> np.ndarray:
220         if target_day < 7:
221             raise ValueError('递归次日预测至少需要7天历史数据。')
222         next_date = dates[target_day] + timedelta(days=1)
223         slot = np.arange(PERIODS, dtype=float)
224         midpoint_hour = (slot * 10.0 + 5.0) / 60.0
225         tod_angle = 2.0 * np.pi * midpoint_hour / 24.0
226         doy_angle = 2.0 * np.pi * next_date.timetuple().tm_yday / 365.0
227         dow = next_date.weekday()
228         hist3 = np.vstack((series_kw[target_day - 2:target_day], current_prediction))
229         hist7 = np.vstack((series_kw[target_day - 6:target_day], current_prediction))
230         X = np.empty((PERIODS, 13), dtype=float)
231         X[:, 0] = slot
232         X[:, 1] = np.sin(tod_angle)
233         X[:, 2] = np.cos(tod_angle)
234         X[:, 3] = float(dow)
235         X[:, 4] = 1.0 if dow >= 5 else 0.0
236         X[:, 5] = math.sin(doy_angle)
237         X[:, 6] = math.cos(doy_angle)

```

```

238     X[:, 7] = current_prediction
239     X[:, 8] = series_kw[target_day - 6]
240     X[:, 9] = np.mean(hist3, axis=0)
241     X[:, 10] = np.mean(hist7, axis=0)
242     X[:, 11] = np.std(hist7, axis=0)
243     X[:, 12] = float(next_date.month)
244     return X
245
246 def predict_two_day(self, target_day: int) -> tuple[np.ndarray, np.ndarray]:
247     load_today, pv_today = self.predict(target_day)
248     assert self.load_model is not None and self.pv_model is not None
249     load_X_next = self._recursive_next_features(self.load_kw, self.dates, target_day,
250     load_today)
251     pv_X_next = self._recursive_next_features(self.pv_kw, self.dates, target_day, pv_today)
252     load_next = np.clip(np.asarray(self.load_model.predict(load_X_next, validate_features=
253     False), dtype=float), 0.0, None)
254     pv_next = np.clip(np.asarray(self.pv_model.predict(pv_X_next, validate_features=False),
255     dtype=float), 0.0, None)
256     return (np.vstack((load_today, load_next)), np.vstack((pv_today, pv_next)))
257
258 def generate_two_day_scenarios(load_pred_kw: np.ndarray, pv_pred_kw: np.ndarray,
259     residual_history: list[ForecastRecord], scenario_days: int) -> tuple[np.ndarray, np.ndarray
260     , tuple[str, ...]]:
261     if load_pred_kw.shape != (2, PERIODS) or pv_pred_kw.shape != (2, PERIODS):
262         raise ValueError('两日预测矩阵必须为(2,144)。')
263     valid = [r for r in residual_history if r.load_error_kw is not None and r.pv_error_kw is
264     not None]
265     pairs = [(a, b) for a, b in zip(valid[:-1], valid[1:]) if (b.date_value - a.date_value).
266     days == 1]
267     if pairs:
268         selected = pairs[-scenario_days:]
269     elif valid:
270         selected = [(valid[-1], valid[-1])]
271     else:
272         return (load_pred_kw.reshape(1, 2 * PERIODS).copy(), pv_pred_kw.reshape(1, 2 * PERIODS)
273         .copy(), ())
274     load_s = np.empty((len(selected), 2 * PERIODS), dtype=float)
275     pv_s = np.empty_like(load_s)
276     source_dates: list[str] = []
277     for s, (first, second) in enumerate(selected):
278         assert first.load_error_kw is not None and first.pv_error_kw is not None
279         assert second.load_error_kw is not None and second.pv_error_kw is not None
280         load_s[s] = np.concatenate((np.clip(load_pred_kw[0] + first.load_error_kw, 0.0, None),
281         np.clip(load_pred_kw[1] + second.load_error_kw, 0.0, None)))
282         pv_s[s] = np.concatenate((np.clip(pv_pred_kw[0] + first.pv_error_kw, 0.0, None), np.
283         clip(pv_pred_kw[1] + second.pv_error_kw, 0.0, None)))
284         source_dates.append(f'{first.date_value.isoformat()}+{second.date_value.isoformat()}')
285     return (load_s, pv_s, tuple(source_dates))

```

```

276
277 def terminal_energy_value(price: np.ndarray, params: ModelParameters) -> float:
278     daily_price = np.asarray(price, dtype=float).reshape(-1)[:PERIODS]
279     return float(params.terminal_value_scale * params.discharge_efficiency * np.median(
        daily_price))
280
281 def solve_stochastic_plan(price: np.ndarray, scenario_load_kw: np.ndarray, scenario_pv_kw: np.
    ndarray, params: ModelParameters, initial_energy: float, daily_cycle: bool,
    terminal_value_per_kwh: float | None=None) -> DayPlan:
282     S, T = scenario_load_kw.shape
283     if T < PERIODS or T % PERIODS != 0 or scenario_pv_kw.shape != (S, T):
284         raise ValueError('场景矩阵维度错误。')
285     price = np.asarray(price, dtype=float).reshape(-1)
286     if price.size == PERIODS and T > PERIODS:
287         price = np.tile(price, T // PERIODS)
288     if price.size != T:
289         raise ValueError('电价向量长度必须与滚动窗口时段数一致。')
290     if terminal_value_per_kwh is None:
291         terminal_value_per_kwh = terminal_energy_value(price, params)
292     scenario_net = (scenario_load_kw - scenario_pv_kw) * DT_HOURS
293     prob = np.full(S, 1.0 / S)
294     G_START = 0
295     SCENARIO_SIZE = 5 * T + 1
296     n_vars = T + S * SCENARIO_SIZE
297
298     def offsets(s: int) -> tuple[int, int, int, int, int]:
299         base = T + s * SCENARIO_SIZE
300         c = base
301         d = c + T
302         h = d + T
303         w = h + T
304         e = w + T
305         return (c, d, h, w, e)
306     a_eq = lil_matrix((S * 2 * T, n_vars), dtype=float)
307     b_eq = np.zeros(S * 2 * T, dtype=float)
308     for s in range(S):
309         c0, d0, h0, w0, e0 = offsets(s)
310         for t in range(T):
311             r_bal = s * 2 * T + t
312             a_eq[r_bal, G_START + t] = 1.0
313             a_eq[r_bal, c0 + t] = -1.0
314             a_eq[r_bal, d0 + t] = 1.0
315             a_eq[r_bal, h0 + t] = 1.0
316             a_eq[r_bal, w0 + t] = -1.0
317             b_eq[r_bal] = scenario_net[s, t]
318             r_state = s * 2 * T + T + t
319             a_eq[r_state, e0 + t] = -1.0
320             a_eq[r_state, e0 + t + 1] = 1.0

```

```

321         a_eq[r_state, c0 + t] = -params.charge_efficiency
322         a_eq[r_state, d0 + t] = 1.0 / params.discharge_efficiency
323     bounds: list[tuple[float | None, float | None]] = []
324     bounds.extend([(0.0, None)] * T)
325     for s in range(S):
326         bounds.extend([(0.0, params.max_energy_per_period)] * T)
327         bounds.extend([(0.0, params.max_energy_per_period)] * T)
328         bounds.extend([(0.0, None)] * T)
329         bounds.extend([(0.0, None)] * T)
330         bounds.append((initial_energy, initial_energy))
331         bounds.extend([(params.capacity_min, params.capacity_max)] * (T - 1))
332         bounds.append((params.capacity_min, params.capacity_max))
333     cost = np.zeros(n_vars, dtype=float)
334     cost[G_START:G_START + T] = price
335     for s in range(S):
336         _, _, h0, _, e0 = offsets(s)
337         cost[h0:h0 + T] = prob[s] * params.emergency_multiplier * price
338         cost[e0 + T] = -prob[s] * terminal_value_per_kwh
339     primary = linprog(cost, A_eq=a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method='highs')
340     if not primary.success:
341         raise RuntimeError(f'随机购电主优化失败: {primary.message}')
342     throughput = np.zeros(n_vars, dtype=float)
343     for s in range(S):
344         c0, d0, _, _, _ = offsets(s)
345         throughput[c0:c0 + T] = prob[s]
346         throughput[d0:d0 + T] = prob[s]
347     tolerance = max(1e-06, abs(primary.fun) * 1e-09)
348     a_ub = csr_matrix(cost.reshape(1, -1))
349     secondary = linprog(throughput, A_ub=a_ub, b_ub=np.asarray([primary.fun + tolerance]), A_eq
350 =a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method='highs')
351     result = secondary if secondary.success else primary
352     x = np.where(np.abs(result.x) < EPS_ZERO, 0.0, result.x)
353     full_grid = x[:T].copy()
354     grid = full_grid[:PERIODS].copy()
355     return DayPlan(grid=grid, preview_grid=full_grid[PERIODS:].copy())
356
357 def solve_actual_dispatch(price: np.ndarray, grid_plan: np.ndarray, actual_load_kw: np.ndarray,
358     actual_pv_kw: np.ndarray, params: ModelParameters, initial_energy: float, daily_cycle:
359 bool, *, preview_grid: np.ndarray | None=None, preview_load_kw: np.ndarray | None=None,
360     preview_pv_kw: np.ndarray | None=None, terminal_value_per_kwh: float | None=None) ->
361     ActualDispatch:
362     price = np.asarray(price, dtype=float).reshape(-1)
363     grid_plan = np.asarray(grid_plan, dtype=float).reshape(-1)
364     actual_load_kw = np.asarray(actual_load_kw, dtype=float).reshape(-1)
365     actual_pv_kw = np.asarray(actual_pv_kw, dtype=float).reshape(-1)
366     if any((x.size != PERIODS for x in (price, grid_plan, actual_load_kw, actual_pv_kw))):
367         raise ValueError('实际日内调度的当日电价、合同、负荷和光伏均须含144个时段。')
368     has_preview = preview_grid is not None

```

```

364 if has_preview != (preview_load_kw is not None) or has_preview != (preview_pv_kw is not
    None):
365     raise ValueError('次日预览的购电、负荷和光伏必须同时提供。')
366 if has_preview:
367     assert preview_grid is not None and preview_load_kw is not None and (preview_pv_kw is
    not None)
368     preview_grid = np.asarray(preview_grid, dtype=float).reshape(-1)
369     preview_load_kw = np.asarray(preview_load_kw, dtype=float).reshape(-1)
370     preview_pv_kw = np.asarray(preview_pv_kw, dtype=float).reshape(-1)
371     if any((x.size != PERIODS for x in (preview_grid, preview_load_kw, preview_pv_kw))):
372         raise ValueError('次日预览向量均须含144个时段。')
373     grid_full = np.concatenate((grid_plan, preview_grid))
374     load_full = np.concatenate((actual_load_kw, preview_load_kw))
375     pv_full = np.concatenate((actual_pv_kw, preview_pv_kw))
376     price_full = np.tile(price, 2)
377 else:
378     grid_full = grid_plan
379     load_full = actual_load_kw
380     pv_full = actual_pv_kw
381     price_full = price
382 T = grid_full.size
383 net = (load_full - pv_full) * DT_HOURS
384 if terminal_value_per_kwh is None:
385     terminal_value_per_kwh = terminal_energy_value(price, params)
386 c0 = 0
387 d0 = T
388 h0 = 2 * T
389 w0 = 3 * T
390 e0 = 4 * T
391 n_vars = 5 * T + 1
392 a_eq = lil_matrix((2 * T, n_vars), dtype=float)
393 b_eq = np.zeros(2 * T, dtype=float)
394 for t in range(T):
395     a_eq[t, c0 + t] = -1.0
396     a_eq[t, d0 + t] = 1.0
397     a_eq[t, h0 + t] = 1.0
398     a_eq[t, w0 + t] = -1.0
399     b_eq[t] = net[t] - grid_full[t]
400     r = T + t
401     a_eq[r, e0 + t] = -1.0
402     a_eq[r, e0 + t + 1] = 1.0
403     a_eq[r, c0 + t] = -params.charge_efficiency
404     a_eq[r, d0 + t] = 1.0 / params.discharge_efficiency
405 bounds: list[tuple[float | None, float | None]] = []
406 bounds.extend([(0.0, params.max_energy_per_period)] * T)
407 bounds.extend([(0.0, params.max_energy_per_period)] * T)
408 bounds.extend([(0.0, None)] * T)
409 bounds.extend([(0.0, None)] * T)

```

```

410     bounds.append((initial_energy, initial_energy))
411     bounds.extend([(params.capacity_min, params.capacity_max)] * (T - 1))
412     bounds.append((params.capacity_min, params.capacity_max))
413     emergency_obj = np.zeros(n_vars, dtype=float)
414     emergency_obj[h0:h0 + T] = params.emergency_multiplier * price_full
415     emergency_obj[e0 + T] = -terminal_value_per_kwh
416     primary = linprog(emergency_obj, A_eq=a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method='highs')
417     if not primary.success:
418         raise RuntimeError(f'实际日内调度失败: {primary.message}')
419     secondary_obj = np.zeros(n_vars, dtype=float)
420     secondary_obj[c0:c0 + T] = 1.0
421     secondary_obj[d0:d0 + T] = 1.0
422     secondary_obj[w0:w0 + T] = 1e-06
423     tolerance = max(1e-06, abs(primary.fun) * 1e-09)
424     secondary = linprog(secondary_obj, A_ub=csr_matrix(emergency_obj.reshape(1, -1)), b_ub=np.
425         asarray([primary.fun + tolerance]), A_eq=a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method='
426         highs')
427     result = secondary if secondary.success else primary
428     x = np.where(np.abs(result.x) < EPS_ZERO, 0.0, result.x)
429     C_full = x[c0:c0 + T]
430     D_full = x[d0:d0 + T]
431     H_full = x[h0:h0 + T]
432     W_full = x[w0:w0 + T]
433     E_full = x[e0:e0 + T + 1]
434     C = C_full[:PERIODS]
435     D = D_full[:PERIODS]
436     H = H_full[:PERIODS]
437     W = W_full[:PERIODS]
438     E = E_full[:PERIODS + 1]
439     balance_residual = grid_plan + D + H - C - W - net[:PERIODS]
440     state_residual = E[1:] - E[:-1] - params.charge_efficiency * C + D / params.
441     discharge_efficiency
442     max_balance_error = float(np.max(np.abs(balance_residual)))
443     max_state_error = float(np.max(np.abs(state_residual)))
444     if max(max_balance_error, max_state_error) > 1e-05:
445         raise RuntimeError('实际调度未通过供需平衡或储能状态残差检查。')
446     emergency_cost = float(params.emergency_multiplier * price @ H)
447     planned_cost = float(price @ grid_plan)
448     return ActualDispatch(charge=C, discharge=D, emergency=H, waste=W, energy=E, emergency_cost
449         =emergency_cost, total_cost=planned_cost + emergency_cost, used_secondary=secondary.success
450         , max_balance_error=max_balance_error, max_state_error=max_state_error)
451
452 def validate_cross_day_results(results: list[DayResult], params: ModelParameters) -> None:
453     if not results:
454         raise ValueError('没有可写入的日结果。')
455     for i, result in enumerate(results):
456         energy = np.asarray(result.dispatch.energy, dtype=float)

```

```

452     if energy.size != PERIODS + 1 or not np.all(np.isfinite(energy)):
453         raise RuntimeError(f'{result.date_value} 的SOC轨迹维度或数值异常。')
454     if energy.min() < params.capacity_min - 1e-05 or energy.max() > params.capacity_max + 1
e-05:
455         raise RuntimeError(f'{result.date_value} 的SOC越过安全边界。')
456     if i == 0:
457         continue
458     previous = results[i - 1]
459     if (result.date_value - previous.date_value).days != 1:
460         raise RuntimeError('输出日期不连续，无法验证跨日SOC传递。')
461     if abs(previous.dispatch.energy[-1] - energy[0]) > 1e-05:
462         raise RuntimeError(f'{previous.date_value} 24:00与{result.date_value} 0:00的SOC不连
续。')
463
464 def run_problem2(data, params, cfg, start_date, end_date, daily_cycle=False, verbose=False):
465     dates = {d: i for i, d in enumerate(data.dates)}
466     start, end = (dates[start_date], dates[end_date])
467     if start < 14 or end < start:
468         raise ValueError('日期区间无效')
469     forecaster = RollingForecaster(data.load_kw, data.pv_kw, data.dates, cfg)
470     history = []
471     results = []
472     energy = params.initial_energy
473     value = terminal_energy_value(data.price, params)
474     for d in range(max(cfg.residual_warmup_start_day, 14), end + 1):
475         load, pv = forecaster.predict_two_day(d)
476         sl, sp, sources = generate_two_day_scenarios(load, pv, history, cfg.scenario_days)
477         plan = solve_stochastic_plan(np.tile(data.price, 2), sl, sp, params, energy, False,
value)
478         dispatch = solve_actual_dispatch(data.price, plan.grid, data.load_kw[d], data.pv_kw[d],
params, energy, False, preview_grid=plan.preview_grid, preview_load_kw=load[1],
preview_pv_kw=pv[1], terminal_value_per_kwh=value)
479         if d >= start:
480             results.append(DayResult(date_value=data.dates[d], plan=plan, dispatch=dispatch))
481             history.append(ForecastRecord(date_value=data.dates[d], load_error_kw=data.load_kw[d] -
load[0], pv_error_kw=data.pv_kw[d] - pv[0]))
482             energy = float(dispatch.energy[-1])
483     return (results, {})
484
485 def save_results(results, metrics, data, output):
486     output.mkdir(parents=True, exist_ok=True)
487     labels = physical_interval_labels()
488     with (output / '计划购电量.csv').open('w', encoding='utf-8-sig', newline='') as f:
489         w = csv.writer(f)
490         w.writerow(['日期', *labels, '全天购电量_kWh', '全天购电费_元'])
491         for r in results:
492             w.writerow([r.date_value, *r.plan.grid, r.plan.grid.sum(), data.price @ r.plan.grid
])

```

```

493 with (output / '充放电量.csv').open('w', encoding='utf-8-sig', newline='') as f:
494     w = csv.writer(f)
495     w.writerow(['日期', '时段', '充电_kWh', '放电_kWh', '储电时刻', '储电量_kWh'])
496     for r in results:
497         d = r.dispatch
498         for i in range(6):
499             a, b = (24 * i, 24 * (i + 1))
500             w.writerow([r.date_value, f'{4 * i}:00-{4 * i + 4}:00', d.charge[a:b].sum(), d.
discharge[a:b].sum(), '0:00' if i == 0 else '24:00' if i == 1 else '', d.energy[0] if i ==
0 else d.energy[-1] if i == 1 else ''])
501
502 def events(values):
503     rows = []
504     start = None
505     for t, flag in enumerate(np.r_[values > EMERGENCY_THRESHOLD, False]):
506         if flag and start is None:
507             start = t
508         elif not flag and start is not None:
509             rows.append((f'{format_clock_from_minutes(start * 10)}-{
format_clock_from_minutes(t * 10)}', float(values[start:t].sum())))
510             start = None
511     return rows or [('无', 0.0)]
512 with (output / '紧急购电量.csv').open('w', encoding='utf-8-sig', newline='') as f:
513     w = csv.writer(f)
514     w.writerow(['日期', '紧急购电时段', '紧急购电量_kWh'])
515     for r in results:
516         w.writerows(((r.date_value, label, amount) for label, amount in events(r.dispatch.
emergency)))
517 lookup = {r.date_value: r for r in results}
518 for day in [date(2025, 3, 20), date(2025, 6, 21), date(2025, 9, 23), date(2025, 12, 21)]:
519     if day not in lookup:
520         print(day, '未计算')
521         continue
522     r = lookup[day]
523     d = r.dispatch
524     print('\n', day, '表1: 指定时段购电量 (kWh) ')
525     for h in [10, 12, 14, 16, 18, 20]:
526         print(labels[h * 6], f'{r.plan.grid[h * 6]:.4f}')
527     print(f'全天购电量: {r.plan.grid.sum():.4f} kWh; 全天购电费: {data.price @ r.plan.grid:.4
f} 元')
528     print('表2: 分段充电量、放电量 (kWh) ')
529     for i in range(6):
530         print(f'{4 * i}:00-{4 * i + 4}:00', f'{d.charge[24 * i:24 * (i + 1)].sum():.4f}', f
'{d.discharge[24 * i:24 * (i + 1)].sum():.4f}')
531     print(f'0:00电量: {d.energy[0]:.4f}; 24:00电量: {d.energy[-1]:.4f} kWh')
532     print('表3: 紧急购电量 (kWh) ')
533     for label, amount in events(d.emergency):
534         print(label, f'{amount:.4f}')

```

```

535     print('结果目录: ', output.resolve())
536
537 def main():
538     base = Path(__file__).resolve().parent
539     p = argparse.ArgumentParser(description='问题二附录核心程序, 无可可视化')
540     p.add_argument('--price', type=Path, default=base / '附件1.xlsx')
541     p.add_argument('--actual', type=Path, default=base / '附件2.xlsx')
542     p.add_argument('--start', default='2025-02-01')
543     p.add_argument('--end', default='2025-12-31')
544     p.add_argument('--output-dir', type=Path, default=base / '问题二_题目要求结果')
545     p.add_argument('--scenario-days', type=int, default=30)
546     p.add_argument('--n-estimators', type=int, default=180)
547     p.add_argument('--retrain-every', type=int, default=1)
548     p.add_argument('--terminal-value-scale', type=float, default=1.0)
549     args = p.parse_args()
550     if min(args.scenario_days, args.n_estimators, args.retrain_every) < 1 or args.
        terminal_value_scale < 0:
551         raise ValueError('参数取值无效')
552     params = ModelParameters(terminal_value_scale=args.terminal_value_scale)
553     cfg = ForecastConfig(scenario_days=args.scenario_days, n_estimators=args.n_estimators,
        retrain_every=args.retrain_every)
554     data = load_year_data(args.price, args.actual)
555     results, metrics = run_problem2(data, params, cfg, date.fromisoformat(args.start), date.
        fromisoformat(args.end), False, False)
556     validate_cross_day_results(results, params)
557     save_results(results, metrics, data, args.output_dir)
558 if __name__ == '__main__':
559     main()

```

1.1.3 问题三求解代码

Listing 3 问题三求解程序

```

1  """问题三求解程序。
2  读取附件数据后完成负荷预测、场景构造与SAA-MPC滚动优化，
3  并统计计划购电、调整费用、紧急购电及SOC等结果。
4  采用0:00制定初始计划、6:00和12:00滚动更新的策略。
5  """
6  from __future__ import annotations
7  import argparse
8  import sys
9  from dataclasses import dataclass
10 from datetime import date, datetime, time
11 from pathlib import Path
12 sys.dont_write_bytecode = True
13 import numpy as np
14 import openpyxl

```

```

15 from openpyxl import Workbook
16 from scipy.optimize import linprog
17 from scipy.sparse import csr_matrix, lil_matrix
18 BASE_DIR = Path(__file__).resolve().parent
19 PERIODS = 144
20 DT_HOURS = 1.0 / 6.0
21 UPDATES = (6, 12)
22 POLICY_LABEL = '0:00+6:00+12:00'
23 EPS = 1e-08
24 EMERGENCY_THRESHOLD = 1e-06
25 @dataclass(frozen=True)
26 class ModelParameters:
27     capacity_min: float = 1200.0; capacity_max: float = 10800.0
28     initial_soc: float = 6000.0; max_power_kw: float = 5000.0
29     charge_efficiency: float = 0.9; discharge_efficiency: float = 0.9
30     emergency_multiplier: float = 5.0; increase_multiplier: float = 1.5
31     cancellation_refund_multiplier: float = 0.5
32     residual_days: int = 28; k_scenarios: int = 7; load_weight_window: int = 28
33     lookahead_days: int = 1; terminal_value_scale: float = 1.0
34     @property
35     def max_energy_per_slot(self) -> float:
36         return self.max_power_kw * DT_HOURS
37 def terminal_energy_value(price: np.ndarray, params: ModelParameters) -> float:
38     return float(params.terminal_value_scale * params.discharge_efficiency * np.median(np.
39         asarray(price, dtype=float)))
40 @dataclass(frozen=True)
41 class ProblemData:
42     dates: list[date]; price: np.ndarray
43     load_kw: np.ndarray; pv_kw: np.ndarray
44     pv_hourly: dict[date, dict[int, np.ndarray]]
45 @dataclass(frozen=True)
46 class ScenarioSet:
47     residual_kw: np.ndarray; probabilities: np.ndarray
48 @dataclass(frozen=True)
49 class StagePlan:
50     grid: np.ndarray; charge: np.ndarray; discharge: np.ndarray
51 @dataclass(frozen=True)
52 class DayRun:
53     date_value: date
54     initial_grid: np.ndarray; final_grid: np.ndarray
55     emergency: np.ndarray; soc: np.ndarray
56     initial_plan_cost: float; adjustment_net_cost: float; emergency_cost: float
57     total_cost: float; emergency_kwh: float; waste_kwh: float
58 def contract_labels() -> list[str]:
59     return [f'{format_clock(10 + i * 10)}--{format_clock(20 + i * 10)}'
60         for i in range(PERIODS)]
61 def format_clock(minutes: int) -> str:
62     if minutes == 1440:

```

```

62     return '0:00+1'
63 if minutes == 1450:
64     return '0:10+1'
65 if minutes > 1440:
66     h, m = divmod(minutes - 1440, 60)
67     return f'{h}:{m:02d}+1'
68 h, m = divmod(minutes, 60)
69 return f'{h}:{m:02d}'
70 def parse_date_like(value: object) -> date:
71     if isinstance(value, datetime):
72         return value.date()
73     if isinstance(value, date):
74         return value
75     text = str(value).strip().replace('/', '-').replace('.', '-').split()[0]
76     return date(*(int(x) for x in text.split('-')))
77 def parse_release_hour(value: object) -> int:
78     if isinstance(value, datetime):
79         return value.hour
80     if isinstance(value, time):
81         return value.hour
82     return int(str(value).strip().split(':', 1)[0])
83 # 读取题目附件并整理基础数据
84 def read_price(path: Path) -> np.ndarray:
85     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
86     rows = list(wb.active.iter_rows(min_row=2, values_only=True)); wb.close()
87     price = np.asarray([r[1] for r in rows], dtype=float)
88     if price.shape != (PERIODS,) or not np.all(np.isfinite(price)):
89         raise ValueError('附件1必须包含144个有效电价。')
90     return price
91 def read_actual_year(path: Path) -> tuple[list[date], np.ndarray, np.ndarray]:
92     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
93     required = ['小区负载', '光伏发电实际功率']
94     for name in required:
95         if name not in wb.sheetnames:
96             wb.close()
97             raise ValueError(f'附件2缺少工作表: {name}')
98     load_rows = list(wb[required[0]].iter_rows(values_only=True))
99     pv_rows = list(wb[required[1]].iter_rows(values_only=True)); wb.close()
100    dates = [parse_date_like(r[0]) for r in load_rows[1:]]
101    pv_dates = [parse_date_like(r[0]) for r in pv_rows[1:]]
102    if dates != pv_dates:
103        raise ValueError('附件2负载与光伏日期不一致。')
104    load = np.asarray([r[1:] for r in load_rows[1:]], dtype=float)
105    pv = np.asarray([r[1:] for r in pv_rows[1:]], dtype=float)
106    if load.shape != pv.shape or load.shape[1] != PERIODS or not np.all(np.isfinite([load, pv]
107    )):
108        raise ValueError('附件2负载和光伏必须同形且各含144个有效时段值。')
109    return dates, load, pv

```

```

109 def read_pv_hourly_forecasts(path: Path) -> dict[date, dict[int, np.ndarray]]:
110     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
111     rows = list(wb.active.iter_rows(min_row=2, values_only=True)); wb.close()
112     result: dict[date, dict[int, np.ndarray]] = {}
113     current_date: date | None = None
114     for row in rows:
115         if row[0] not in (None, ''):
116             current_date = parse_date_like(row[0])
117             if current_date is None:
118                 continue
119             release = parse_release_hour(row[1])
120             values = np.asarray(row[2:26], dtype=float)
121             if values.size != 24 or not np.all(np.isfinite(values)):
122                 raise ValueError(f'附件3 {current_date} {release}:00 必须含24个有效值。')
123             result.setdefault(current_date, {})[release] = np.clip(values, 0.0, None)
124     return result
125
126 def load_problem_data(price_path: Path, actual_path: Path, pv_forecast_path: Path) ->
127     ProblemData:
128     price = read_price(price_path)
129     dates, load_kw, pv_kw = read_actual_year(actual_path)
130     pv_hourly = read_pv_hourly_forecasts(pv_forecast_path)
131     return ProblemData(dates, price, load_kw, pv_kw, pv_hourly)
132
133 # 根据历史数据拟合负荷预测权重
134
135 def fit_simplex_weights(X: np.ndarray, y: np.ndarray) -> np.ndarray:
136     if X.size == 0 or y.size == 0:
137         return np.full(3, 1.0 / 3.0)
138     candidates: list[np.ndarray] = []
139     H, b = X.T @ X, X.T @ y
140     KKT = np.block([[H, np.ones((3, 1))], [np.ones((1, 3)), np.zeros((1, 1))]])
141     try:
142         sol = np.linalg.lstsq(KKT, np.r_[b, 1.0], rcond=None)[0][:3]
143         if np.all(sol >= -1e-10):
144             sol = np.clip(sol, 0.0, None)
145             candidates.append(sol / sol.sum())
146     except np.linalg.LinAlgError:
147         pass
148     for k in range(3):
149         ij = [i for i in range(3) if i != k]
150         a, bidx = ij; z = X[:, a] - X[:, bidx]; target = y - X[:, bidx]
151         denom = float(z @ z)
152         t = float(z @ target / denom) if denom > 1e-14 else 0.5
153         t = min(1.0, max(0.0, t))
154         w = np.zeros(3, dtype=float); w[a] = t; w[bidx] = 1.0 - t
155         candidates.append(w)
156     candidates.extend([np.eye(3)[i] for i in range(3)])
157     return np.asarray(min(candidates, key=lambda w: float(np.sum((X @ w - y) ** 2))), dtype=
158         float)
159
160 def build_load_forecasts(data: ProblemData, params: ModelParameters) -> np.ndarray:

```

```

155     n_days = len(data.dates)
156     pred = np.full_like(data.load_kw, np.nan, dtype=float)
157     for d in range(n_days):
158         if d < 21:
159             continue
160         train_days = list(range(max(21, d - params.load_weight_window), d))
161         if train_days:
162             X_parts = []
163             y_parts = []
164             for j in train_days:
165                 Xj = np.column_stack([data.load_kw[j - 7], data.load_kw[j - 14], data.load_kw[j
- 21]])
166                 X_parts.append(Xj)
167                 y_parts.append(data.load_kw[j])
168                 w = fit_simplex_weights(np.vstack(X_parts), np.concatenate(y_parts))
169             else:
170                 w = np.full(3, 1.0 / 3.0)
171                 pred[d] = w[0] * data.load_kw[d - 7] + w[1] * data.load_kw[d - 14] + w[2] * data.
load_kw[d - 21]
172                 pred[d] = np.clip(pred[d], 0.0, None)
173     return pred
174 def interpolate_pv_forecast(data: ProblemData, day_idx: int, release_hour: int) -> np.ndarray:
175     d = data.dates[day_idx]
176     releases = data.pv_hourly.get(d, {})
177     if release_hour not in releases:
178         raise ValueError(f'附件3缺少 {d} {release_hour}:00 预报。')
179     hourly = releases[release_hour]
180     release_min = release_hour * 60
181     anchor = (max(0.0, data.pv_kw[day_idx - 1, -1]) if day_idx else 0.0) if release_hour == 0
else max(0.0, data.pv_kw[day_idx, release_hour * 6 - 1])
182     x = np.asarray([release_min] + [release_min + 60 * k for k in range(1, 25)], dtype=float)
183     y = np.concatenate([[anchor], hourly.astype(float)])
184     q = np.arange(1, PERIODS + 1, dtype=float) * 10.0
185     forecast = np.interp(q, x, y)
186     forecast[q < release_min] = np.nan
187     return np.clip(forecast, 0.0, None)
188 # 使用k-medoids从历史残差中提取代表场景
189 def pairwise_euclidean(X: np.ndarray) -> np.ndarray:
190     sq = np.sum(X * X, axis=1, keepdims=True)
191     dist2 = np.maximum(sq + sq.T - 2.0 * X @ X.T, 0.0)
192     return np.sqrt(dist2)
193 def k_medoids(X: np.ndarray, k: int, max_iter: int=50) -> tuple[np.ndarray, np.ndarray]:
194     n = X.shape[0]
195     if n == 0:
196         raise ValueError('k-medoids输入为空。')
197     k = max(1, min(k, n))
198     D = pairwise_euclidean(X)
199     medoids = [int(np.argmin(D.sum(axis=1)))]

```

```

200 while len(medoids) < k:
201     nearest = np.min(D[:, medoids], axis=1)
202     nearest[medoids] = -1.0
203     medoids.append(int(np.argmax(nearest)))
204 medoids = np.asarray(medoids, dtype=int)
205 labels = np.argmin(D[:, medoids], axis=1)
206 for _ in range(max_iter):
207     new_medoids = medoids.copy()
208     for c in range(k):
209         members = np.where(labels == c)[0]
210         if members.size == 0:
211             continue
212         sub = D[np.ix_(members, members)]
213         new_medoids[c] = members[int(np.argmin(sub.sum(axis=1)))]
214 new_medoids = np.asarray(sorted(set(new_medoids.tolist())), dtype=int)
215 while new_medoids.size < k:
216     nearest = np.min(D[:, new_medoids], axis=1)
217     nearest[new_medoids] = -1.0
218     candidate = int(np.argmax(nearest))
219     new_medoids = np.append(new_medoids, candidate)
220 new_labels = np.argmin(D[:, new_medoids], axis=1)
221 if np.array_equal(np.sort(medoids), np.sort(new_medoids)) and np.array_equal(labels,
new_labels):
222     medoids = new_medoids
223     labels = new_labels
224     break
225 medoids = new_medoids
226 labels = new_labels
227 return (medoids, labels)
228 def build_scenario_set(data: ProblemData, load_forecast_kw: np.ndarray, target_day: int,
release_hour: int, params: ModelParameters) -> ScenarioSet:
229 start_hist = max(21, target_day - params.residual_days)
230 residuals: list[np.ndarray] = []
231 active_start = release_hour * 6
232 for h in range(start_hist, target_day):
233     if not np.all(np.isfinite(load_forecast_kw[h])):
234         continue
235     try:
236         pv_pred = interpolate_pv_forecast(data, h, release_hour)
237     except (ValueError, KeyError):
238         continue
239 forecast_net = load_forecast_kw[h] - pv_pred
240 actual_net = data.load_kw[h] - data.pv_kw[h]
241 residual = actual_net - forecast_net
242 residual[:active_start] = 0.0
243 if not np.all(np.isfinite(residual[active_start:])):
244     continue
245 residuals.append(residual)

```

```

246     if not residuals:
247         return ScenarioSet(np.zeros((1, PERIODS), dtype=float), np.ones(1, dtype=float))
248     R = np.vstack(residuals)
249     segment = R[:, active_start:]
250     scale = float(np.std(segment))
251     X = segment / scale if scale > 1e-09 else segment.copy()
252     medoid_rows, labels = k_medoids(X, params.k_scenarios)
253     probs = np.asarray([(labels == c).mean() for c in range(len(medoid_rows))], dtype=float)
254     return ScenarioSet(R[medoid_rows], probs)
255 def build_tail_net_forecast(data: ProblemData, day_idx: int, lookahead_days: int,
256     fallback_net_kw: np.ndarray, start_slot: int=0) -> np.ndarray:
257     days = max(0, int(lookahead_days))
258     if days == 0:
259         return np.empty(0, dtype=float)
260     if not 0 <= int(start_slot) < PERIODS:
261         raise ValueError('start_slot必须位于[0,143]。')
262     fallback = np.asarray(fallback_net_kw, dtype=float)
263     if fallback.shape != (PERIODS,) or not np.all(np.isfinite(fallback)):
264         raise ValueError('跨日前视fallback净负荷必须是144个有限值。')
265     tail_slots = days * PERIODS + int(start_slot)
266     required_future_days = days + (1 if start_slot else 0)
267     pieces: list[np.ndarray] = []
268     for step in range(1, required_future_days + 1):
269         target_idx = day_idx + step
270         hist_idx = [target_idx - lag for lag in (7, 14, 21) if 0 <= target_idx - lag <= day_idx
271 ]
272         if hist_idx:
273             net_hat = np.mean(data.load_kw[hist_idx], axis=0) - np.mean(data.pv_kw[hist_idx],
274 axis=0)
275         else:
276             net_hat = fallback.copy()
277         pieces.append(np.asarray(net_hat, dtype=float))
278     return np.concatenate(pieces)[:tail_slots]
279 # 单次滚动阶段的SAA-MPC优化
280 def solve_stage(*, price: np.ndarray, base_net_kw: np.ndarray, scenarios: ScenarioSet, params:
281     ModelParameters, start_slot: int, start_soc: float, previous_grid: np.ndarray | None,
282     previous_charge: np.ndarray | None, previous_discharge: np.ndarray | None, tail_net_kw: np.
283     ndarray | None=None) -> StagePlan:
284     n = PERIODS - start_slot
285     if n <= 0:
286         raise ValueError('start_slot越界。')
287     active = slice(start_slot, PERIODS); p = price[active]
288     base = np.asarray(base_net_kw[active], dtype=float)
289     R = np.asarray(scenarios.residual_kw[:, active], dtype=float)
290     probs = np.asarray(scenarios.proBABILITIES, dtype=float); K = R.shape[0]
291     scenario_net_kwh = (base[None, :] + R) * DT_HOURS
292     tail = np.asarray(tail_net_kw, dtype=float).reshape(-1) if tail_net_kw is not None else np.
293     empty(0, dtype=float)

```

```

287 m = int(tail.size); tail_net_kwh = tail * DT_HOURS
288 tail_price = np.resize(np.asarray(price, dtype=float), m) if m else np.empty(0)
289 is_update = previous_grid is not None
290 g0 = 0; c0 = g0 + n; d0 = c0 + n; s0 = d0 + n; cursor = s0 + n + 1
291 if is_update:
292     ap0 = cursor; am0 = ap0 + n; cursor = am0 + n
293 else:
294     ap0 = am0 = -1
295 h_starts = [cursor + 2 * n * k for k in range(K)]
296 w_starts = [h + n for h in h_starts]; cursor += 2 * n * K
297 if m:
298     tg0 = cursor; tc0 = tg0 + m; td0 = tc0 + m
299     th0 = td0 + m; tw0 = th0 + m; ts0 = tw0 + m; cursor = ts0 + m + 1
300 else:
301     tg0 = tc0 = td0 = th0 = tw0 = ts0 = -1
302 n_vars = cursor
303 eq_rows = K * n + n + (n if is_update else 0) + (1 + 2 * m if m else 0)
304 Aeq = lil_matrix((eq_rows, n_vars), dtype=float)
305 beq = np.zeros(eq_rows, dtype=float); row = 0
306 for k in range(K):
307     h0, w0 = h_starts[k], w_starts[k]
308     for i in range(n):
309         Aeq[row, g0 + i] = 1.0; Aeq[row, d0 + i] = 1.0
310         Aeq[row, c0 + i] = -1.0; Aeq[row, h0 + i] = 1.0; Aeq[row, w0 + i] = -1.0
311         beq[row] = scenario_net_kwh[k, i]
312         row += 1
313 for i in range(n):
314     Aeq[row, s0 + i] = -1.0; Aeq[row, s0 + i + 1] = 1.0
315     Aeq[row, c0 + i] = -params.charge_efficiency; Aeq[row, d0 + i] = 1.0 / params.
316     discharge_efficiency
317     row += 1
318 old_active = None
319 if is_update:
320     old_active = np.asarray(previous_grid[active], dtype=float)
321     for i in range(n):
322         Aeq[row, g0 + i] = 1.0; Aeq[row, ap0 + i] = -1.0; Aeq[row, am0 + i] = 1.0
323         beq[row] = old_active[i]
324         row += 1
325 if m:
326     Aeq[row, ts0] = 1.0
327     Aeq[row, s0 + n] = -1.0
328     row += 1
329     for i in range(m):
330         Aeq[row, tg0 + i] = 1.0; Aeq[row, td0 + i] = 1.0
331         Aeq[row, th0 + i] = 1.0; Aeq[row, tc0 + i] = -1.0; Aeq[row, tw0 + i] = -1.0
332         beq[row] = tail_net_kwh[i]
333         row += 1
334     for i in range(m):

```

```

334         Aeq[row, ts0 + i] = -1.0; Aeq[row, ts0 + i + 1] = 1.0
335         Aeq[row, tc0 + i] = -params.charge_efficiency; Aeq[row, td0 + i] = 1.0 / params.
discharge_efficiency
336         row += 1
337     assert row == eq_rows
338     zero, power = (0.0, None), (0.0, params.max_energy_per_slot)
339     soc_bound = (params.capacity_min, params.capacity_max)
340     bounds = [zero] * n + [power] * (2 * n) + [(start_soc, start_soc)] + [soc_bound] * n
341     if is_update:
342         assert old_active is not None
343         bounds += [zero] * n + [(0.0, float(v)) for v in old_active]
344     bounds += [zero] * (2 * K * n)
345     if m:
346         bounds += [zero] * m + [power] * (2 * m) + [zero] * (2 * m) + [soc_bound] * (m + 1)
347     primary_obj = np.zeros(n_vars, dtype=float)
348     if is_update:
349         primary_obj[ap0:ap0 + n] = params.increase_multiplier * p
350         primary_obj[am0:am0 + n] = -params.cancellation_refund_multiplier * p
351     else:
352         primary_obj[g0:g0 + n] = p
353     for k in range(K):
354         primary_obj[h_starts[k]:h_starts[k] + n] = probs[k] * params.emergency_multiplier * p
355     if m:
356         primary_obj[tg0:tg0 + m] = tail_price
357         primary_obj[th0:th0 + m] = params.emergency_multiplier * tail_price
358     terminal_soc_index = ts0 + m if m else s0 + n
359     primary_obj[terminal_soc_index] -= terminal_energy_value(price, params)
360     solve_obj = primary_obj.copy(); tie = 1e-08
361     solve_obj[c0:c0 + n] += tie; solve_obj[d0:d0 + n] += tie
362     for k in range(K):
363         solve_obj[w_starts[k]:w_starts[k] + n] += probs[k] * tie * 0.001
364     if m:
365         solve_obj[tc0:tc0 + m] += tie; solve_obj[td0:td0 + m] += tie
366         solve_obj[tw0:tw0 + m] += tie * 0.001
367     primary = linprog(solve_obj, A_eq=Aeq.tocsr(), b_eq=beq, bounds=bounds, method='highs')
368     if not primary.success:
369         raise RuntimeError(f'SAA-MPC主优化失败(start={start_slot}): {primary.message}')
370     x = np.where(np.abs(primary.x) < EPS, 0.0, primary.x)
371     if previous_grid is None:
372         full_g = np.zeros(PERIODS); full_c = np.zeros(PERIODS); full_d = np.zeros(PERIODS)
373     else:
374         full_g = np.asarray(previous_grid, dtype=float).copy()
375         full_c = np.asarray(previous_charge, dtype=float).copy()
376         full_d = np.asarray(previous_discharge, dtype=float).copy()
377     full_g[active] = x[g0:g0 + n]
378     full_c[active] = x[c0:c0 + n]
379     full_d[active] = x[d0:d0 + n]
380     return StagePlan(full_g, full_c, full_d)

```

```

381 def execute_segment(*, data: ProblemData, day_idx: int, params: ModelParameters, grid: np.
    ndarray, charge: np.ndarray, discharge: np.ndarray, start_slot: int, end_slot: int, soc: np
    .ndarray, emergency: np.ndarray, waste: np.ndarray) -> None:
382     actual_net_kwh = (data.load_kw[day_idx] - data.pv_kw[day_idx]) * DT_HOURS
383     for t in range(start_slot, end_slot):
384         available = grid[t] + discharge[t] - charge[t]
385         gap = actual_net_kwh[t] - available
386         emergency[t] = max(0.0, gap)
387         waste[t] = max(0.0, -gap)
388         soc[t + 1] = soc[t] + params.charge_efficiency * charge[t] - discharge[t] / params.
            discharge_efficiency
389         if soc[t + 1] < params.capacity_min - 1e-05 or soc[t + 1] > params.capacity_max + 1e
            -05:
390             raise RuntimeError(f'{data.dates[day_idx]} 时段{t}实际执行SOC越界: {soc[t + 1]}')
391 # 按日执行初始计划及日内滚动更新
392 def run_one_day(*, data: ProblemData, load_forecast_kw: np.ndarray, params: ModelParameters,
    day_idx: int, scenario_cache: dict[tuple[int, int], ScenarioSet], start_soc: float | None=
    None, updates: tuple[int, ...]=UPDATES) -> DayRun:
393     start_soc = params.initial_soc if start_soc is None else float(start_soc)
394     d = data.dates[day_idx]
395     if not np.all(np.isfinite(load_forecast_kw[day_idx])):
396         raise ValueError(f'{d} 的负荷历史不足, 无法进行21日滞后预测。')
397     def scenarios_for(release: int) -> ScenarioSet:
398         key = (day_idx, release)
399         if key not in scenario_cache:
400             scenario_cache[key] = build_scenario_set(data, load_forecast_kw, day_idx, release,
                params)
401         return scenario_cache[key]
402     net0 = load_forecast_kw[day_idx] - interpolate_pv_forecast(data, day_idx, 0)
403     tail_net = build_tail_net_forecast(data, day_idx, params.lookahead_days, net0, start_slot
        =0)
404     plan = solve_stage(price=data.price, base_net_kw=net0, scenarios=scenarios_for(0), params=
        params, start_slot=0, start_soc=start_soc, previous_grid=None, previous_charge=None,
        previous_discharge=None, tail_net_kw=tail_net)
405     initial_grid = plan.grid.copy()
406     current_grid = plan.grid.copy()
407     current_c = plan.charge.copy()
408     current_d = plan.discharge.copy()
409     emergency = np.zeros(PERIODS, dtype=float)
410     waste = np.zeros(PERIODS, dtype=float)
411     soc = np.full(PERIODS + 1, np.nan, dtype=float)
412     soc[0] = start_soc
413     adjustment_cost = 0.0
414     executed_until = 0
415     for release in updates:
416         start_slot = release * 6
417         execute_segment(data=data, day_idx=day_idx, params=params, grid=current_grid, charge=
            current_c, discharge=current_d, start_slot=executed_until, end_slot=start_slot, soc=soc,

```

```

emergency=emergency, waste=waste)
418     executed_until = start_slot
419     old_grid = current_grid.copy()
420     old_c = current_c.copy()
421     old_d = current_d.copy()
422     net_new = load_forecast_kw[day_idx] - interpolate_pv_forecast(data, day_idx, release)
423     rolling_tail = build_tail_net_forecast(data, day_idx, params.lookahead_days, net0,
start_slot=start_slot)
424     new_plan = solve_stage(price=data.price, base_net_kw=net_new, scenarios=scenarios_for(
release), params=params, start_slot=start_slot, start_soc=float(soc[start_slot]),
previous_grid=old_grid, previous_charge=old_c, previous_discharge=old_d, tail_net_kw=
rolling_tail)
425     diff = new_plan.grid[start_slot:] - old_grid[start_slot:]
426     plus = np.maximum(diff, 0.0)
427     minus = np.maximum(-diff, 0.0)
428     p = data.price[start_slot:]
429     adjustment_cost += float(params.increase_multiplier * p @ plus - params.
cancellation_refund_multiplier * p @ minus)
430     current_grid = new_plan.grid
431     current_c = new_plan.charge
432     current_d = new_plan.discharge
433     execute_segment(data=data, day_idx=day_idx, params=params, grid=current_grid, charge=
current_c, discharge=current_d, start_slot=executed_until, end_slot=PERIODS, soc=soc,
emergency=emergency, waste=waste)
434     initial_cost = float(data.price @ initial_grid)
435     emergency_cost = float(params.emergency_multiplier * data.price @ emergency)
436     total_cost = initial_cost + adjustment_cost + emergency_cost
437     return DayRun(d, initial_grid, current_grid.copy(), emergency, soc,
438                 initial_cost, adjustment_cost, emergency_cost,
439                 total_cost, float(emergency.sum()), float(waste.sum()))
440 def run_days(data: ProblemData, load_forecast_kw: np.ndarray, params: ModelParameters,
441             start_idx: int, end_idx: int, scenario_cache: dict, initial_soc: float,
442             previous: DayRun | None=None, updates: tuple[int, ...]=UPDATES):
443     energy, results = float(initial_soc), []
444     for day_idx in range(start_idx, end_idx + 1):
445         r = run_one_day(data=data, load_forecast_kw=load_forecast_kw, params=params,
446                        day_idx=day_idx, scenario_cache=scenario_cache,
447                        start_soc=energy, updates=updates)
448         if previous is not None and abs(r.soc[0] - previous.soc[-1]) > 1e-6:
449             raise RuntimeError('相邻日期SOC不连续。')
450         results.append(r)
451         previous, energy = r, float(r.soc[-1])
452     return results, previous, energy
453 def summarize_policy(results: list[DayRun]) -> dict[str, float]:
454     soc_all = np.concatenate([r.soc for r in results])
455     end_soc = np.asarray([r.soc[-1] for r in results])
456     continuity_error = max((abs(results[i].soc[0] - results[i - 1].soc[-1]) for i in range(1,
len(results)))), default=0.0)

```

```

457 return {'days': float(len(results)), 'start_soc_kwh': float(results[0].soc[0]), '
end_soc_kwh': float(results[-1].soc[-1]), 'min_soc_kwh': float(np.min(soc_all)), '
max_soc_kwh': float(np.max(soc_all)), 'soc_continuity_error_kwh': float(continuity_error),
'end_near_min_days': float(np.sum(end_soc <= 1201.0)), 'end_near_max_days': float(np.sum(
end_soc >= 10799.0)), 'initial_plan_cost_yuan': float(sum((r.initial_plan_cost for r in
results))), 'adjustment_net_cost_yuan': float(sum((r.adjustment_net_cost for r in results))
), 'emergency_cost_yuan': float(sum((r.emergency_cost for r in results))), 'total_cost_yuan
': float(sum((r.total_cost for r in results))), 'emergency_kwh': float(sum((r.emergency_kwh
for r in results))), 'waste_kwh': float(sum((r.waste_kwh for r in results)))}
458 def add_matrix_sheet(wb: Workbook, title: str, results: list[DayRun], getter,
459 labels: list[str]) -> None:
460     ws = wb.create_sheet(title)
461     ws.append(["日期", *labels])
462     for r in results:
463         ws.append([r.date_value.isoformat(), *map(float, getter(r))])
464     ws.freeze_panes = "B2"
465     for row in ws.iter_rows(min_row=2, min_col=2):
466         for cell in row:
467             cell.number_format = "0.0000"
468 # 汇总并导出论文附录所需结果
469 def write_appendix_result(path: Path, results: list[DayRun]) -> None:
470     path.parent.mkdir(parents=True, exist_ok=True)
471     wb = Workbook()
472     ws = wb.active
473     ws.title = "逐日汇总"
474     ws.append(["日期", "日初SOC/kWh", "日末SOC/kWh", "初始计划费/元",
475 "调整净费用/元", "紧急购电费/元", "总购电费用/元",
476 "紧急购电量/kWh", "弃电量/kWh"])
477     for r in results:
478         ws.append([r.date_value.isoformat(), float(r.soc[0]), float(r.soc[-1]),
479 r.initial_plan_cost, r.adjustment_net_cost, r.emergency_cost,
480 r.total_cost, r.emergency_kwh, r.waste_kwh])
481     ws.freeze_panes = "A2"
482     for row in ws.iter_rows(min_row=2, min_col=2):
483         for cell in row:
484             cell.number_format = "0.0000"
485     labels = contract_labels()
486     add_matrix_sheet(wb, "初始计划购电", results, lambda r: r.initial_grid, labels)
487     add_matrix_sheet(wb, "调整后合同购电", results, lambda r: r.final_grid, labels)
488     add_matrix_sheet(wb, "紧急购电", results, lambda r: r.emergency, labels)
489     add_matrix_sheet(wb, "SOC轨迹", results, lambda r: r.soc,
490 [f"E{i}" for i in range(PERIODS + 1)])
491     wb.save(path)
492 def print_key_results(summary: dict[str, float], results: list[DayRun], output: Path) -> None:
493     print("\n问题3核心计算完成")
494     print(f"策略: {POLICY_LABEL}; 回测天数: {int(summary['days'])}")
495     print(f"购电费用: 初始计划 {summary['initial_plan_cost_yuan']:.2f} 元, "
496 f"调整净费用 {summary['adjustment_net_cost_yuan']:.2f} 元, "

```

```

497         f"紧急购电 {summary['emergency_cost_yuan']:.2f} 元, "
498         f"合计 {summary['total_cost_yuan']:.2f} 元")
499     print(f"紧急购电量: {summary['emergency_kwh']:.4f} kWh; "
500           f"弃电量: {summary['waste_kwh']:.4f} kWh")
501     print(f"SOC: {summary['start_soc_kwh']:.2f} -> {summary['end_soc_kwh']:.2f} kWh; "
502           f"范围 [{summary['min_soc_kwh']:.2f}, {summary['max_soc_kwh']:.2f}] kWh; "
503           f"跨日连续误差 {summary['soc_continuity_error_kwh']:.3e} kWh")
504     by_day = {r.date_value: r for r in results}
505     print("指定日期: 合同购电量 / 紧急购电量 / 总购电费用 / 日初→日末SOC")
506     for day in (date(2025, 3, 20), date(2025, 6, 21), date(2025, 9, 23), date(2025, 12, 21)):
507         r = by_day.get(day)
508         if r is None:
509             print(f" {day}: 未计算")
510         else:
511             print(f" {day}: {r.final_grid.sum():.2f} / {r.emergency_kwh:.2f} kWh; "
512                   f"{r.total_cost:.2f} 元; {r.soc[0]:.2f}→{r.soc[-1]:.2f} kWh")
513     print(f"结果文件: {output.resolve()}")
514 def parse_args() -> argparse.Namespace:
515     parser = argparse.ArgumentParser(description="问题3论文附录核心程序（无可可视化）")
516     parser.add_argument("--price", type=Path, default=BASE_DIR.parent / "附件1.xlsx")
517     parser.add_argument("--actual", type=Path, default=BASE_DIR.parent / "附件2.xlsx")
518     parser.add_argument("--pv-forecast", type=Path, default=BASE_DIR.parent / "附件3.xlsx")
519     parser.add_argument("--output", type=Path, default=BASE_DIR / "result3_appendix.xlsx")
520     parser.add_argument("--start-date", default="2025-02-01")
521     parser.add_argument("--end-date", default="2025-12-31")
522     return parser.parse_args()
523 def main() -> None:
524     args = parse_args()
525     params = ModelParameters()
526     data = load_problem_data(args.price, args.actual, args.pv_forecast)
527     load_forecast = build_load_forecasts(data, params)
528     start_day, end_day = date.fromisoformat(args.start_date), date.fromisoformat(args.end_date)
529     if start_day not in data.dates or end_day not in data.dates:
530         raise ValueError("起止日期必须存在于附件2。")
531     start_idx, end_idx = data.dates.index(start_day), data.dates.index(end_day)
532     if start_idx < 22 or end_idx < start_idx:
533         raise ValueError("评价期至少从第23个样本日开始，且结束日期不得早于开始日期。")
534     cache: dict[tuple[int, int], ScenarioSet] = {}
535     _, previous, start_soc = run_days(data, load_forecast, params, 21, start_idx - 1,
536                                       cache, params.initial_soc, updates=())
537     results, _, _ = run_days(data, load_forecast, params, start_idx, end_idx,
538                             cache, start_soc, previous)
539     summary = summarize_policy(results)
540     write_appendix_result(args.output, results)
541     print_key_results(summary, results, args.output)
542 if __name__ == "__main__":
543     main()

```

1.1.4 问题四求解代码

Listing 4 问题四求解程序

```
1 """问题四求解程序。
2 读取实际负荷、光伏预报与实时电价数据，分别完成问题4-2和问题4-3的滚动优化计算，
3 并输出年度费用汇总及指定日期结果。问题4-3采用0:00制定计划、6:00和12:00更新的策略。
4 """
5 from __future__ import annotations
6
7 import argparse
8 import sys
9 from datetime import date
10 from pathlib import Path
11
12 sys.dont_write_bytecode = True
13 import problem4 as model
14
15 BASE_DIR = Path(__file__).resolve().parent
16 ROOT = BASE_DIR.parent
17 POLICY = "0+6+12"
18
19
20 # 读取运行参数及输入、输出文件位置
21 def parse_args() -> argparse.Namespace:
22     p = argparse.ArgumentParser(description="问题4附录精简程序（无绘图）")
23     p.add_argument("--actual", type=Path, default=ROOT / "附件2.xlsx")
24     p.add_argument("--pv-forecast", type=Path, default=ROOT / "附件3.xlsx")
25     p.add_argument("--dynamic-price", type=Path, default=ROOT / "附件4.xlsx")
26     p.add_argument("--template42", type=Path, default=ROOT / "附件5" / "result4-2.xlsx")
27     p.add_argument("--template43", type=Path, default=ROOT / "附件5" / "result4-3.xlsx")
28     p.add_argument("--output42", type=Path, default=BASE_DIR / "result4-2.xlsx")
29     p.add_argument("--output43", type=Path, default=BASE_DIR / "result4-3.xlsx")
30     p.add_argument("--start", default="2025-02-01")
31     p.add_argument("--end", default="2025-12-31")
32     return p.parse_args()
33
34
35 # 保存问题4-2的计算结果
36 def save_q42(args: argparse.Namespace, results, accuracy, params) -> None:
37     template = args.template42 if args.template42.exists() else None
38     if not model.try_fill_q42_template(template, args.output42, results, accuracy, params):
39         model.create_q42_workbook(args.output42, results, accuracy, params)
40
41
42 # 保存问题4-3的计算结果
43 def save_q43(args: argparse.Namespace, results, summary, accuracy, params) -> None:
44     summaries = {POLICY: summary}
```

```

45     template = args.template43 if args.template43.exists() else None
46     if not model.try_fill_q43_template(
47         template, args.output43, results, summaries, accuracy, [], params
48     ):
49         model.create_q43_workbook(args.output43, results, summaries, accuracy, [], params)
50
51
52 # 输出年度汇总及指定日期结果
53 def print_results(q42, q43, s42: dict, s43: dict, args: argparse.Namespace) -> None:
54     print("\n问题4核心计算完成")
55     print(
56         f"Q4-2: 计划费 {s42['planned_cost_yuan']:.2f} 元, "
57         f"紧急购电费 {s42['emergency_cost_yuan']:.2f} 元, "
58         f"总费用 {s42['total_cost_yuan']:.2f} 元"
59     )
60     print(
61         f"Q4-3 ({model.POLICY_LABELS[POLICY]}): 初始计划费 "
62         f"{s43['initial_plan_cost_yuan']:.2f} 元, 调整净费用 "
63         f"{s43['adjustment_net_cost_yuan']:.2f} 元, 紧急购电费 "
64         f"{s43['emergency_cost_yuan']:.2f} 元, 总费用 {s43['total_cost_yuan']:.2f} 元"
65     )
66     q42_by_day = {r.date_value: r for r in q42}
67     q43_by_day = {r.date_value: r for r in q43}
68     print("指定日期: Q4-2总费用 / Q4-3总费用 (元) ")
69     for d in map(date.fromisoformat, ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")):
70         a, b = q42_by_day.get(d), q43_by_day.get(d)
71         left = f"{a.dispatch.total_cost:.2f}" if a else "未计算"
72         right = f"{b.total_cost:.2f}" if b else "未计算"
73         print(f" {d}: {left} / {right}")
74     print(f"结果文件: \n {args.output42.resolve()}\n {args.output43.resolve()}")
75
76
77 # 主流程: 数据读取、预测、滚动优化与结果导出
78 def main() -> None:
79     args = parse_args()
80     params = model.ModelParameters(
81         k_scenarios=7, residual_days=28, lookahead_days=1, terminal_value_scale=1.0
82     )
83     data = model.load_problem_data(args.actual, args.pv_forecast, args.dynamic_price)
84     load_pred = model.build_load_forecasts(data, params)
85     price_pred = model.build_price_base_forecasts(data, params)
86     day_index = {d: i for i, d in enumerate(data.dates)}
87     start, end = date.fromisoformat(args.start), date.fromisoformat(args.end)
88     if start not in day_index or end not in day_index:
89         raise ValueError("起止日期必须同时存在于附件2和附件4。")
90     first, last = day_index[start], day_index[end]
91     if first < 22 or first > last:
92         raise ValueError("评价期至少从第23个样本日开始, 且start不能晚于end。")

```

```

93     cache: dict[tuple[int, int], model.JointScenarioSet] = {}
94
95     q42 = model.run_q42(data, load_pred, price_pred, params, first, last, cache)
96     accuracy = model.price_forecast_accuracy(data, price_pred, params, first, last)
97     s42 = model.summarize_q42(q42)
98     save_q42(args, q42, accuracy, params)
99
100     previous, start_soc = model.build_q43_common_warmup(
101         data, load_pred, price_pred, params, first, cache
102     )
103     q43 = model.run_q43_policy(
104         data, load_pred, price_pred, params, first, last, POLICY, cache,
105         initial_soc=start_soc, initial_previous=previous
106     )
107     s43 = model.summarize_q43(q43)
108     save_q43(args, q43, s43, accuracy, params)
109     print_results(q42, q43, s42, s43, args)
110
111
112 if __name__ == "__main__":
113     main()

```